APU2F2511CS(AI)

CT127-3-2-PFDA – PROGRAMMING FOR DATA ANALYSIS

DATA ANALYSIS GROUP ASSIGNMENT

HAND OUT DATE:	23 NOV 2025
HAND IN DATE:	07 FEB 2026
WEIGHTAGE:	60%

GROUP No: 18

Member 1: Leong Yuan Kang TP076118

Member 2: Lew Li Jun TP087450

Member 3: Carrie Yap Liu Qing TP075883

Member 4: Sherwin A/L Jesudass TP075823

1.0 Introduction

1.1 Project Description

In the contemporary world, web-based services have been rapidly growing accompanied by complicated cyber threats. One of the persistent issue is website defacement, where attackers alter a site's content or appearance without authorization. Although they are often overlooked as digital graffiti, but in many cases, this type of incidents are signs of underlying vulnerability, resulting in a loss of money, reputation and the disruption of operations.

This project, Analysis of Cyber-Attack Events, applies R and RStudio to convert raw website defacement into structural analytic framework. Going beyond the descriptive observation, the research uses statistical analysis and data transformation techniques to reveal the trends in the cyber-attacks all around the world, offering practical insights to support informed cybersecurity decision-making.

The project aims to integrate and cleanse multiple datasets by addressing data quality issues such as missing values, inconsistent character encoding, and extreme outliers. Besides, it also aims to identify geographic and technical patterns by examining the relationship between hosting countries, web server versions, and attack frequency, to help identify high-risk environments. In addition, the study also used to determine the economic impact of cyber-attacks by analyzing the relationships among downtime, ransom demands, and financial losses. Lastly, the fourth objective is that using superior R tools to explore, transform, and visualize data to produce a more in-dept insights than simple descriptive statistics, thus demonstrates analytical depth.

Analysing cyber-attack events is becoming particularly significant in the digital economy where even short periods of downtime can directly cause revenue loss. Gaining visibility into technical vulnerabilities such as outdated or misconfigured web server versions, allows organizations to prioritise defensive measures more effectively. Last but not least, Ransom_Outliers_Flag of the dataset captures the attention of the rare but severe "Black

Swan" incidents. This project will go beyond a reactive mindset by focusing on these high-impact events through R, adopting a proactive and data-driven process which will not only deal with the high frequency but extreme high financial risks of an attack.

1.2 Data Description

“Encoding” variable contain in the dataset represent multiple encoding method such as “utf-8”, “iso-8859-1”, and “window 1252”. Character encoding in the system used to map human readable characters such as letters, numbers and symbols into a binary code that computer can process with it. By having a standardized character encoding algorithm, all the text will be display in a sorted and transmitted way across different systems, language and coding language.

Character encoding play a vital role in cyberattacks and defences activities. By hiding the malicious code into different format, the attacker can easily bypass the antivirus filters due to it appear harmless. A experienced hacker will try to use specific encoding tricks to make the text look legitimate to user while it actually being interpreted differently by the operating system. For example, phishing email may use encoded characters to mimic trusted domains.

1.3 Data Preparation

In this section, we outline the procedures used to import, clean, and prepare the raw data for analysis using the R programming language and the “tidyverse” framework. The data preparation process involved integrating three separate datasets, standardizing variable formats, imputing missing data, and handling outliers to ensure the integrity of the analysis.

1.3.1 Data Loading and Integration

The dataset was split into three parts (“HackingData_Part1.csv”, “HackingData_Part2.xlsx”, “HackingData_Part3.txt”).

First, we utilized the “tidyverse” and “readxl” libraries to load them, Figure 1.3.1.1. After the three dataset is loaded successful, we first check on the first and last 10 columns for each dataset to ensure that the dataset is imported without error or any missing. Figure 1.3.1.2. and Figure 1.3.1.3.

```
# Load 3 dataset
df_part1 <- read_csv("HackingData_Part1.csv", show_col_types = FALSE)
df_part2 <- read_excel("HackingData_Part2.xlsx")
df_part3 <- read_delim("HackingData_Part3.txt", delim = "\t", show_col_types = FALSE)
```

Figure 1.3.1. 1

```
##### Data Exploration & Checking on 3 dataset #####

# View first 10 columns for each dataset
view(head(df_part1, 10))
view(head(df_part2, 10))
view(head(df_part3, 10))

# View last 10 columns for each dataset
view(tail(df_part1, 10))
view(tail(df_part2, 10))
view(tail(df_part3, 10))

# Check data type, size for each dataset
str(df_part1)
str(df_part2)
str(df_part3)
```

Figure 1.3.1. 2

AssignmentScript_Latest.R* | head(df_part1, 10)

Filter

	Date	Notify	URL	IP	Country	WebServer	Encoding	Ransom	DownTime	Loss
1	2/1/1998	Team CodeZero	http://www.janet-jackson.com	NA	UNKNOWN	Unknown	utf-8	NA	18	678
2	3/1/1998	Feliz	http://cariari.ucr.ac.cr	NA	COSTARICA	Unknown	NULL	NA	31	1988
3	4/1/1998	Optiklenz(LOU)	http://marin.k12.ca.us	NA	UNITED STATES	Unknown	iso-8859-1	2034	35	12204
4	4/1/1998	Team CodeZero	http://www.dm.af.mil	NA	AFGHANISTAN	Unknown	utf-8	NA	58	NA
5	4/1/1998	Team CodeZero	http://www.bolling.af.mil	NA	AFGHANISTAN	Unknown	windows-1252	NA	59	NA
6	4/1/1998	foxy man	http://hope.hinet.net	NA	UNKNOWN	Unknown	utf-8	2316	34	13896
7	5/1/1998	Team CodeZero	http://www.nic.ad	NA	ANDORRA	Unknown	utf-8	195	21	390
8	5/1/1998	Zyklon	http://www.tamu-commerce.edu	NA	NA	NA	NULL	NA	37	1895
9	5/1/1998	net_	http://pr0n.prizon.net	NA	UNKNOWN	Unknown	utf-8	2877	28	11508
10	7/1/1998	D.A.M.M.	http://www.unicef.org	NA	UNKNOWN	Unknown	windows-1252	1070	21	2140

Figure 1.3.1. 3

To ensure consistency before merging, we defined a custom function called "standardize_cols". This function converted specific columns ("Date", "Notify", "URL", etc.) to character type and removed non-numeric characters from the numerical columns

("Ransom", "DownTime", "Loss") to ensure clean arithmetic operations later. We then combined the three datasets into a single dataframe using "bind_rows" and formatted the "Date" column using "parse_date_time" to ensure all dates followed the same foramt, Figure 1.3.1.4.

```
##### Standardize Column Types #####
standardize_cols <- function(df) {

  # Column type
  char_cols <- c("Date", "Notify", "URL", "IP", "Country", "WebServer", "Encoding")
  num_cols  <- c("Ransom", "DownTime", "Loss")

  df %>%
    mutate(across(all_of(char_cols), as.character)) %>%
    mutate(across(all_of(num_cols), ~{

      # Keep only digit and decimal points only
      as.numeric(str_replace_all(as.character(.), "[^0-9]", ""))
    }))
}

# Apply standardization & Merge
df_list <- list(df_part1, df_part2, df_part3)
cleaned_list <- map(df_list, standardize_cols)
df <- bind_rows(cleaned_list)


# Format Date columnn
df <- df %>%
  mutate(Date = parse_date_time(Date, orders = c("dmy", "ymd"))) %>%
  mutate(Date = as.Date(Date))
```

Figure 1.3.1. 4

Finally, we checked the dimensions and data types of the combined dataset to confirm that the integration was successful, Figure 1.3.1.5, Figure 1.3.1.6.

```
> dim(df) # Dimension of the dataset (Number of rows and columns)
[1] 592765     10
> str(df)
tibble [592,765 × 10] (S3: tbl_df/tbl/data.frame)
 $ Date     : chr [1:592765] "2/1/1998" "3/1/1998" "4/1/1998" "4/1/1998" ...
 $ Notify   : chr [1:592765] "Team CodeZero" "Feliz" "Optiklenz(LOU)" "Team CodeZero" ...
 $ URL      : chr [1:592765] "http://www.janet-jackson.com" "http://cariari.ucr.ac.cr" "http://marin.k12.ca.us" "http://www.dm.af.mil" ...
 $ IP       : chr [1:592765] NA NA NA NA ...
 $ Country  : chr [1:592765] "UNKNOWN" "COSTARICA" "UNITED STATES" "AFGHANISTAN" ...
 $ WebServer: chr [1:592765] "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding : chr [1:592765] "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Ransom   : num [1:592765] NA NA 2034 NA NA ...
 $ DownTime : num [1:592765] 18 31 35 58 59 34 21 37 28 21 ...
 $ Loss     : num [1:592765] 678 1988 12204 NA NA ...
```

Figure 1.3.1. 5

AssignmentScript_Latest.R* | df

Filter

	Date	Notify	URL	IP	Country	WebServer	Encoding	Ransom	DownTime	Loss
1	1998-01-02	Team CodeZero	http://www.janet-jackson.com	NA	UNKNOWN	Unknown	utf-8	NA	18	678
2	1998-01-03	Feliz	http://cariari.ucr.ac.cr	NA	COSTARICA	Unknown	NULL	NA	31	1988
3	1998-01-04	Optiklenz(LOU)	http://marin.k12.ca.us	NA	UNITED STATES	Unknown	iso-8859-1	2034	35	12204
4	1998-01-04	Team CodeZero	http://www.dm.af.mil	NA	AFGHANISTAN	Unknown	utf-8	NA	58	NA
5	1998-01-04	Team CodeZero	http://www.bolling.af.mil	NA	AFGHANISTAN	Unknown	windows-1252	NA	59	NA
6	1998-01-04	foxy man	http://hope.hinet.net	NA	UNKNOWN	Unknown	utf-8	2316	34	13896
7	1998-01-05	Team CodeZero	http://www.nic.ad	NA	ANDORRA	Unknown	utf-8	195	21	390
8	1998-01-05	Zyklon	http://www.tamu-commerce.edu	NA	NA	NA	NULL	NA	37	1895
9	1998-01-05	net_	http://pr0n.prizon.net	NA	UNKNOWN	Unknown	utf-8	2877	28	11508
10	1998-01-07	D.A.M.M.	http://www.unicef.org	NA	UNKNOWN	Unknown	windows-1252	1070	21	2140
11	1998-01-08	Optiklenz(LOU)	http://www.wvlc.wvnet.edu	NA	UNKNOWN	Unknown	utf-8	2760	33	16560
12	1998-01-09	ViRtU3	http://www.trsystems.com	NA	NA	NA	NULL	NA	17	443
13	1998-01-10	Magica de Bin	http://www.webbnet.com	NA	UNKNOWN	Unknown	utf-8	NA	43	3509
14	1998-01-11	Optiklenz	http://www.easysoftware.com	NA	UNKNOWN	Unknown	utf-8	2406	15	16842
15	1998-01-14	Magica de Bin	http://www.itp.berkeley.edu	NA	UNKNOWN	Unknown	utf-8	902	39	5412
16	1998-01-15	Duncan Silver of L.O.U	http://www.emergent.com	NA	UNKNOWN	Unknown	utf-8	NA	50	2296

Figure 1.3.1. 6

1.3.2 Webserver Version Standardization

After the successful integration of three dataset, to ensure the integrity of the analysis regarding vulnerabilities, the WebServer column have to go through a rigorous standardization process. This step is necessary to remove the malicious noise, formatting error, and metadata that could lead to skew statistical results.

```
> ##############################################################################################
> # WEBSERVER STANDARDIZATION
> ##############################################################################################
> df <- df %>%
+
+   # Clean the WebServer strings (Intermediate step)
+   mutate(
+     Server_Clean = tolower(WebServer),
+     Server_Clean = str_remove_all(Server_Clean, "[\"()]"), # Remove quotes and parentheses
+     Server_Clean = str_trim(Server_Clean)                  # Trim whitespace
+   ) %>%
+
+   # Filter Garbage Rows
+   filter(
+     !str_detect(Server_Clean, "<script>"),    # Remove XSS attacks
+     !str_detect(Server_Clean, "\\*\\*\\*"),   # Remove masked data
+     Server_Clean != "",                       # Remove empty strings
+     !is.na(Server_Clean)
+   ) %>%
+
+   # Finalize: Overwrite original WebServer column with cleaned version
+   mutate(WebServer = Server_Clean) %>%
+
+   # Remove the intermediate column (Keep all other original columns)
+   select(-Server_Clean)
```

First we changed all the value inside the Webserver column to lowercase to eliminate case-sensitivity issues. For the double quotes and parentheses, also be removed to ensure a uniform naming convention.

Not only that, a heuristic noise filtering also performed by separate into 3 ways, security filtering, data masking removal, and integrity check to remove the “garbage” data. First, remove the entire whoever contain <script> tag, because this likely leave by hacker when trying XSS network attack. Second, remove the value who containing masked character (***) as they could not provide any analytical value. Third, remove the empty (NA) value.

And finally, use the cleaned string (Server_Clean) to replace the value inside WebServer to ensure the dataset remain lean and ready for grouping by server type.

1.3.2 Encoding Standardization

In this step, we examined the Encoding column to check for inconsistent entries or typing errors. We generated a frequency table to view the unique encoding types and their counts, Figure 1.3.1.7.

	Encoding	Frequency	Percentage
1	NULL	125080	21.10
2	Windows-1252	76172	12.85
3	UTF-16	76020	12.82
4	ASCII	75943	12.81
5	ISO-8859-1	75539	12.74
6	UTF-8	75035	12.66
7	utf-8	54482	9.19
8	iso-8859-1	9643	1.63
9	windows-1252	9299	1.57
10	ascii	3477	0.59
11	windows-1254	2711	0.46
12	LiteSpeed	2700	0.46
13	iso-8859-9	1194	0.20
14	N	1180	0.20
15	*NA*	904	0.15
16	windows-1256	895	0.15
17	ISO-8859-2	487	0.08
18	windows-1251	282	0.05
19	UTF_7	280	0.05
20	Unknown-Encoding	272	0.05
21	\xFF\xFE	266	0.04

Figure 1.3.1. 7

From the summary table, we observed that some encoding methods were actually the same but were treated as different categories due to case sensitivity. For instance, "Windows-1252" and "windows-1252" refer to the same standard, but the system treated them as separate types because of the lowercase "w".

To fix this, we standardized the entire column by converting the text so that only the first letter is capitalized (using the str_to_sentence function), Figure 1.3.1.8. This cleaning step merged the duplicates successfully. For example, the count for "Windows-1252" increased from 76,172 to 85,471, effectively improving the consistency of our dataset, Figure 1.3.1.9.

```
# Convert all to encoding method name to lowercase
df <- df %>%
  mutate(Encoding = str_to_sentence(Encoding))
```

Figure 1.3.1. 8

	Encoding	Frequency	Percentage
1	Utf-8	129517	21.85
2	Null	125080	21.10
3	Windows-1252	85471	14.42
4	Iso-8859-1	85182	14.37
5	Ascii	79420	13.40
6	Utf-16	76022	12.82
7	Windows-1254	2711	0.46
8	Litespeed	2700	0.46
9	Iso-8859-9	1194	0.20
10	N	1180	0.20
11	*NA*	904	0.15
12	Windows-1256	895	0.15
13	Iso-8859-2	575	0.10
14	Windows-1251	282	0.05
15	Utf_7	280	0.05
16	Unknown-encoding	272	0.05
17	\Xff\xfe	266	0.04
18	Gb2312	262	0.04
19	Utf8\x00	248	0.04
20	Big5	75	0.01
21	Utf-7	43	0.01

Figure 1.3.1. 9

1.3.3 Country Standardization

Next, we inspected the "Country" column and identified several inconsistencies. For example, the United States was represented in multiple ways, such as "Usa", "United State", and "America". Similarly, "Uk" was used instead of "United Kingdom".

To fix this, we first compared our country names against a standard list from the countrycode library to find the mismatches. We then used the case_when function to manually map these variations and misspellings to their official country names (For example, converting "Russian Federation" to "Russia"), Figure 1.3.1.10.

```
# Manual corrections mapping
df <- df %>%
  mutate(
    Country = case_when(
      Country %in% c("Uk", "United Kingd") ~ "United Kingdom",
      Country %in% c("Usa", "United State", "America") ~ "United States",
      Country %in% c("Russian Federation", "Russian Fede", "Russian") ~ "Russia",
      Country == "Viet Nam" ~ "Vietnam",
      Country == "Hong Kong" ~ "Hong Kong SAR China",
      Country == "Czech Republic" ~ "Czechia",
      Country %in% c("Korea", "Korea, Republic of") ~ "South Korea",
      Country %in% c("Newzealand", "New Zealand\"") ~ "New Zealand",
      Country %in% c("saudiarabia", "Saudiarabia") ~ "Saudi Arabia",
      Country == "Costarica" ~ "Costa Rica",
      Country == "Srilanka" ~ "Sri Lanka",
      Country == "Elsalvador" ~ "El Salvador",
      Country == "Burkinafaso" ~ "Burkina Faso",
      Country == "Easttimor" ~ "East Timor",
      Country == "Macedonia" ~ "North Macedonia",
      Country == "Netherlands\"" ~ "Netherlands",
      Country == "Nigeria\"" ~ "Nigeria",
      Country == "Lao People's Democratic Republic" ~ "Laos",

      # Group general region into "Others"
      Country %in% c("Europe", "Asia", "Southamerica", "Westeuro", "Easteuro", "Africa", "Middleeast") ~ "Others",
      TRUE ~ Country
    )
  )

# Consolidate Rare Countries (Frequency < 50)
rare_countries <- df %>%
  count(Country) %>%
  filter(n < 50) %>%
  pull(Country)

# This mutate is change those who with a very low record into Others due to the hidden insight doesnt enough for us to analysis
df <- df %>%
  mutate(
    Country = ifelse(Country %in% rare_countries, "Others", Country)
  )
```

Figure 1.3.1. 10

Finally, to reduce noise in our analysis, we identified countries that appeared very infrequently. Any country with fewer than 50 records was grouped into a generic category

labeled "Others". This helps focus the analysis on the major locations while avoiding clutter from rare values, Figure 1.3.1.11.

	Country	Count	Percentage
1	*NA*	57625	9.7214
2	Others	2072	0.3495
3	European Union	97	0.0164

Figure 1.3.1. 11

1.3.4 Handling Missing Values (Imputation)

By running the “colSums(is.na(df))” function, we can clearly see there are how many NA value in each feature, Figure 1.3.1.12. We analyzed the missing values in “Ransom”, “Loss”, and “DownTime” by checking their skewness and distribution, Figure 1.3.1.13 to .15.

```
> colSums(is.na(df))  # Get total NA value for each feature
     Date    Notify       URL        IP   Country WebServer  Encoding    Ransom  DownTime      Loss
        0      3673      3597     31323     57625      8707       904    162568      2480     36052
```

Figure 1.3.1. 12

```
> # Check skewness & decide strategy
> ransom_skew <- skewness(df$Ransom, na.rm = TRUE)
> message("Ransom Skewness: ", round(ransom_skew, 2))
Ransom Skewness: 5.5
```

Figure 1.3.1. 13

```
> # Check skewness & decide strategy
> loss_skew <- skewness(df$Loss, na.rm = TRUE)
> message("Loss Skewness: ", round(loss_skew, 2))
Loss Skewness: 725.05
```

Figure 1.3.1. 14

```
> message("DownTime Skewness: ", round(downtime_skew, 2))
DownTime Skewness: 40.65
```

Figure 1.3.1. 15

Ransom

We visualized the distributions using density plot, Figure 1.3.1.16 and Figure 1.3.1.17, which revealed high skewness. Our initial plan was to use K-Nearest Neighbors (KNN) imputation. However, as noted in our analysis, the dataset size was too large, causing a memory overflow error (required 521GB RAM), Figure 1.3.1.18. Consequently, we utilized median imputation, which provide a robust replacement value that is not distorted by the high skewness, Figure 1.3.1.19.

```r
# Density plot for Ransom
median_ransom <- median(df$Ransom, na.rm = TRUE)

skew_val <- skewness(df$Ransom, na.rm = TRUE)
skew_desc <- case_when(
  skew_val > 1 ~ "Highly Right Skewed",
  skew_val < -1 ~ "Highly Left Skewed",
  TRUE ~ "Symmetrical"
)

sub_title_ransom <- paste("Skewness:", round(skew_val, 2), "-", skew_desc)

ggplot(df, aes(x = Ransom)) +
  geom_density(fill = "cornflowerblue", alpha = 0.6, color = "darkblue") +

  scale_x_log10(
    breaks = 10^(-1:5),
    labels = comma_format(big.mark = ",")
  ) +
  geom_vline(xintercept = median_ransom, linetype = "dashed", color = "red", linewidth = 0.8) +
  annotate("text", x = median_ransom, y = 0, label = "Median", vjust = -1, color = "red", angle = 90) +
  labs(
    title = "Density Plot of Ransom Amount (Log Scale)",
    subtitle = sub_title_ransom,
    x = "Ransom Amount (Thousands) - Log Scale",
    y = "Density"
  ) +
  theme_minimal()
```

Figure 1.3.1. 16

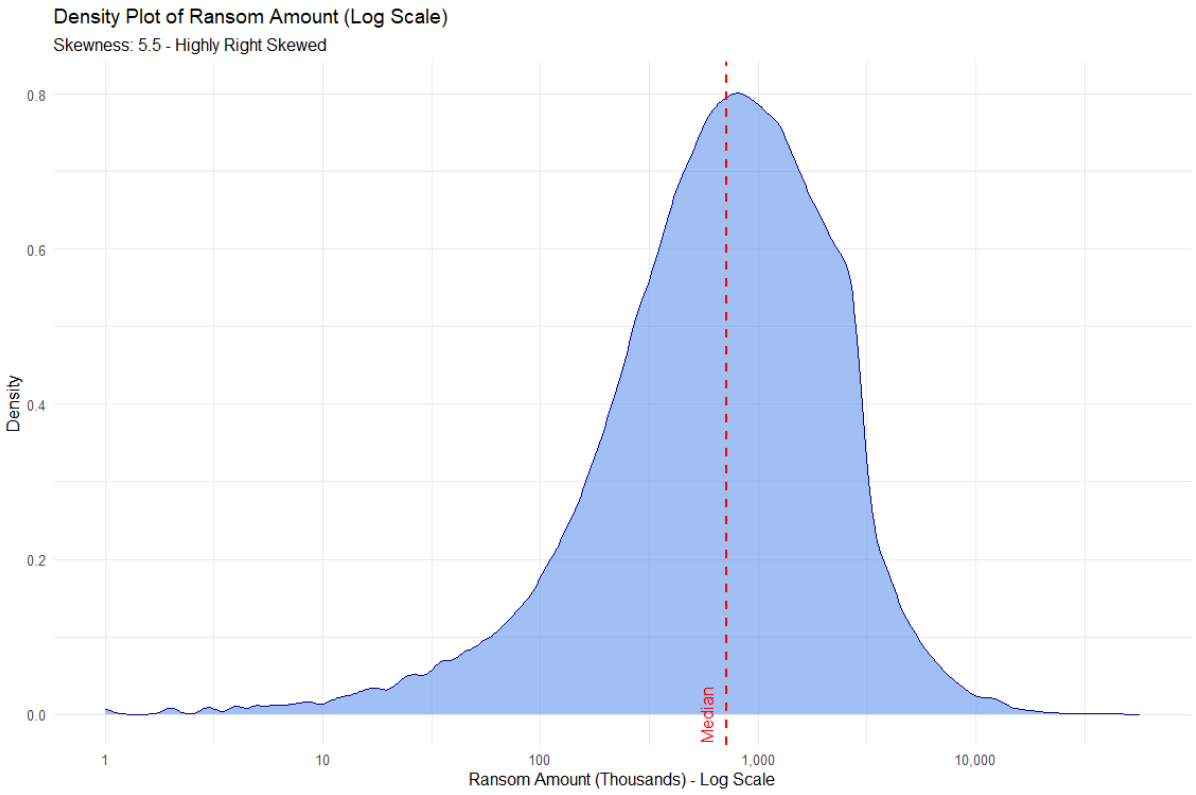


Figure 1.3.1. 17

```
> imputed_subset <- kNN(df[, numeric_cols], variable = c("Ransom"), k = 5, numFun = median, weightDist = TRUE, imp_var = FALSE)
    DownTime         Loss     DownTime         Loss
           0            6         9999 100000000000
错误: cannot allocate vector of size 521.1 Gb
```

Figure 1.3.1. 18

```
ransom_median <- median(df$Ransom, na.rm = TRUE)
df <- df %>%
  mutate(Ransom = replace_na(Ransom, ransom_median))
```

Figure 1.3.1. 19

Loss

“Loss” variable was found to be “Highly Right Skewed”. We visualized this with a histogram on a log scale, Figure 1.3.1.20 and Figure 1.3.1.21. The visualization revealed a bimodal distribution (data with two distinct peaks). For this type of complex distribution, the K-Nearest Neighbors (KNN) imputation method is typically the best approach. However, due to the large size of the dataset, running KNN required more memory (RAM) than out system cloud provide.

```
skew_desc_loss <- case_when(
  loss_skew > 1 ~ "Highly Right Skewed",
  loss_skew < -1 ~ "Highly Left Skewed",
  TRUE ~ "Symmetrical"
)

sub_title_loss <- paste("Skewness:", round(loss_skew, 2), "-", skew_desc_loss)

# Histogram for Loss
ggplot(df %>% filter(Loss > 0), aes(x = Loss)) +
  geom_histogram(bins = 50, fill = "salmon", color = "black", alpha = 0.7) +

  scale_x_log10(breaks = 10^(-1:9), labels = comma_format(big.mark = ",")) +
  scale_y_continuous(labels = comma_format(big.mark = ",")) +

  labs(
    title = "Histogram of Loss Amount (Log Scale)",
    subtitle = sub_title_loss,
    x = "Loss Amount (Thousands - Log Scale)",
    y = "Frequency"
  ) +
  theme_minimal()
```

Figure 1.3.1. 20

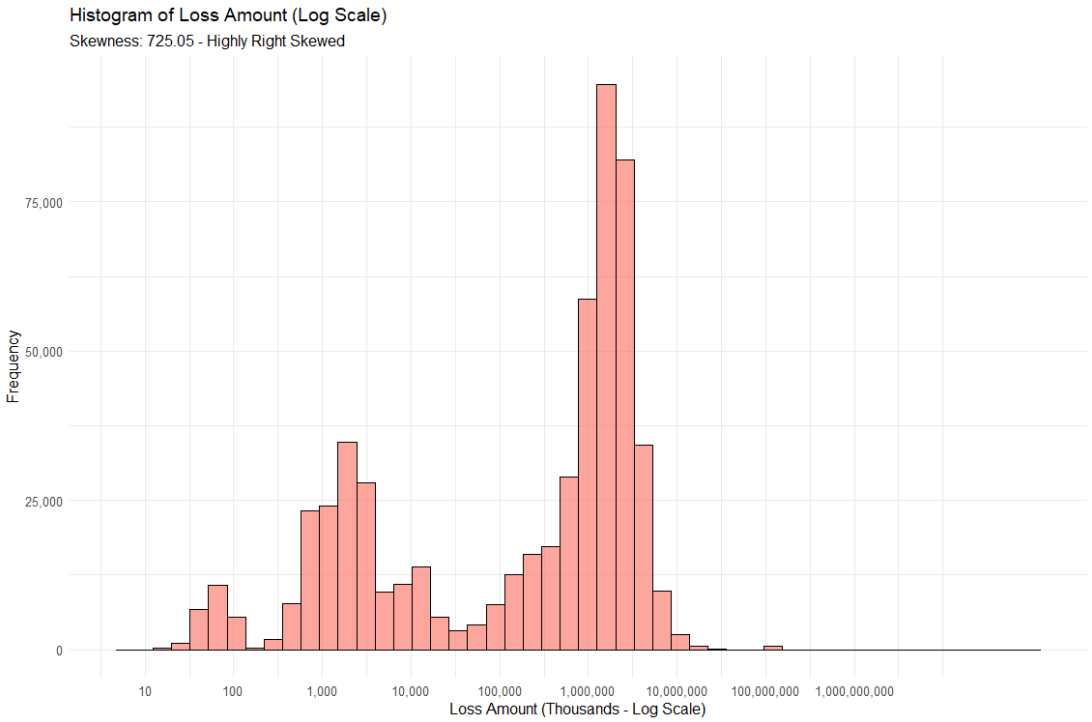


Figure 1.3.1. 21

Given these computational limitations, we opted to use median imputation instead. The median is robust against outliers and skewness, making it suitable alternative for filling the missing value in this context, Figure 1.3.1.22.

```
# Conclusion: Use median to fill up missing value inside the 'Loss' variable

loss_median <- median(df$Loss, na.rm = TRUE)
df <- df %>%
  mutate(Loss = replace_na(Loss, loss_median))
```

Figure 1.3.1. 22

DownTime

we examined the DownTime variable. The analysis revealed a skewness value of 40.65, indicating a highly right-skewed distribution, Figure 1.3.1.23.

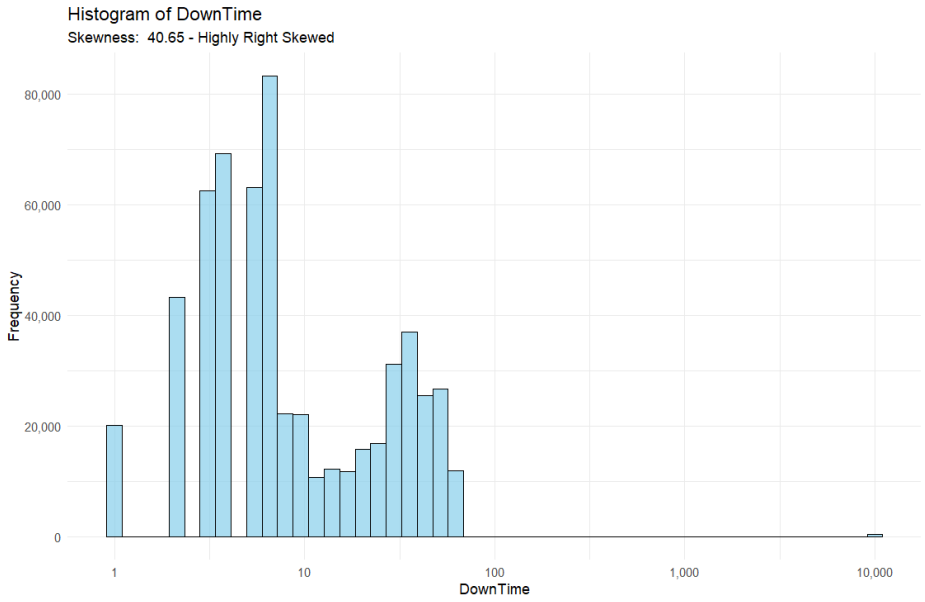


Figure 1.3.1. 23

In highly skewed data, the mean is often distorted by extreme values, so the median is a safer choice for imputation. Consequently, we decided to use the median value to fill in the missing entries. This approach successfully eliminated all missing values in the DownTime column, reducing the missing count from 2,480 to zero.

```
> colSums(is.na(df))
     Date    Notify       URL        IP   Country WebServer  Encoding    Ransom
        0      3673      3597     31323      1559      8707       904         0
 DownTime      Loss
     2480         0
```

Before

```
> downtime_median <- median(df$DownTime, na.rm = TRUE)
> df <- df %>%
+   mutate(DownTime = replace_na(DownTime, downtime_median))
> sum(is.na(df$Loss))
[1] 0
> colSums(is.na(df))
     Date    Notify       URL        IP   Country WebServer  Encoding    Ransom  DownTime      Loss
        0      3673      3597     31323      1559      8707       904         0         0         0
```

After

1.3.5 Data Recovery by URL & IP

To minimize the number of "Unknown" country values, we implemented a two-step recovery process.

URL Recovery

We cleaned the "URL" column to extract the Top-Level Domain (TLD). If the TLD was a specific country code (Example: us, uk), we mapped it to the corresponding country using the "countrycode" library. We ignored generic domain like "com" or "net", Figure 1.3.1.24.

```
# Define generic domain name to ignore
generic_domain <- c("com", "net", "org", "info", "edu", "gov", "biz", "mil", "int", "arpa")

df <- df %>%
  mutate(
    Clean_URL = str_remove_all(str_to_lower(URL), "^https?://|^www\\.|/.*"),  # Remove https://, www\\, and text after last /
    TLD = str_extract(Clean_URL, "[a-z]+$"),
    New_Country = if_else(TLD %in% generic_domain, NA_character_, countrycode(TLD, "iso2c", "country.name", warn = FALSE)),

    # Update Country if Unknown
    Country = case_when(
      (Country %in% c("Unknown", "Others") | is.na(Country)) & !is.na(New_Country) ~ New_Country,
      TRUE ~ Country
    )
  ) %>%
  select(-Clean_URL, -TLD, -New_Country)

colSums(is.na(df))
```

Figure 1.3.1. 24

After run the above code, the number of NA value for Country dropped from previous 57,625 to current 32,966, decreased almost 42.7%. This show that this method is effective and less computational time if compared to the second recovery method.

Before	`> colSums(is.na(df))` Date Notify URL IP Country WebServer Encoding Ransom DownTime Loss 0 3673 3597 31323 57625 8707 904 0 0 0
After	`> colSums(is.na(df))` Date Notify URL IP Country WebServer Encoding Ransom DownTime Loss 0 3673 3597 31323 32966 8707 904 0 0 0

IP Geolocation Recovery

After cleaning the URL data, we noticed that a significant number of rows still had "Unknown" or "Others" in the Country column, but possessed a valid IP address. To recover this missing location data, we implemented an offline IP geolocation method using the *IP2Location Lite* database (CSV format).

We chose this offline approach over an online API because it eliminates connection latency and API rate limits, allowing us to process thousands of IP addresses instantly, Figure 1.3.1.25.

```
csv_file <- "IP2LOCATION-LITE-DB1.CSV"

if (file.exists(csv_file)) {

  # Only read the first 4 line of the csv file
  ip_db <- fread(csv_file, header = FALSE, fill = TRUE, select = c(1:4))
  setnames(ip_db, c("IP_FROM", "IP_TO", "CODE", "COUNTRY_NAME"))

  setkey(ip_db, IP_FROM, IP_TO)

  target_ips_df <- df %>%
    filter((Country %in% c("Unknown", "Others") | is.na(Country)) & !is.na(IP) & IP != "Unknown") %>%
    distinct(IP) %>%
    as.data.table()

  if (nrow(target_ips_df) > 0) {
    target_ips_df <- target_ips_df[grepl("^\\d+\\.\\d+\\.\\d+\\.\\d+$", IP)]

    if (nrow(target_ips_df) > 0) {

      target_ips_df[, c("IP_1", "IP_2", "IP_3", "IP_4") := tstrsplit(IP, ".", fixed=TRUE)]

      target_ips_df[, IP_Num := as.numeric(IP_1) * 16777216 +
                      as.numeric(IP_2) * 65536 +
                      as.numeric(IP_3) * 256 +
                      as.numeric(IP_4)]

      target_ips_df[, c("IP_1", "IP_2", "IP_3", "IP_4") := NULL]


      matched_results <- ip_db[target_ips_df,
                               on = .(IP_FROM <= IP_Num, IP_TO >= IP_Num),
                               .(IP, Found_Country = COUNTRY_NAME)]

      ip_lookup <- as.data.frame(matched_results)

      ip_lookup <- ip_lookup %>% filter(Found_Country != "-" & Found_Country != "Invalid IP Address")

      df <- df %>%
        left_join(ip_lookup, by = "IP") %>%
        mutate(Country = case_when(
          (Country %in% c("Unknown", "Others") | is.na(Country)) & !is.na(Found_Country) ~ Found_Country,
          TRUE ~ Country
        )) %>%
        select(-Found_Country)

      message("SUCCESS")
    } else {
      message("WARNING, FORMAT NOT IPv4, CANT RECOVERY")
    }

  } else {
    message("NO IP ADDRESS NEED TO RECOVER")
  }
} else {
  stop("CANT FOUND THE FOLDER")
}
```

Figure 1.3.1. 25

The recovery process involved three main technical steps:

1. Data Validation (Regex):

First, we filtered the dataset to identify target IPs (where the country was missing). We noticed that some IP entries contained formatting errors or non-standard characters. To ensure accuracy, we applied a Regular Expression (^\\d+\\.\\d+\\.\\d+\\.\\d+$) to strictly retain only valid IPv4 addresses, discarding any malformed data.

2. IP-to-Integer Conversion:

IP geolocation databases store address ranges as integer numbers rather than text to speed up searching. Therefore, we split the IP strings (e.g., "192.168.1.1") into four parts and converted them into a unique numeric value using the standard IPv4 formula:

This conversion allowed us to mathematically compare our IPs against the database ranges.

3. Range Matching and Merging:

Using the data.table library for high-performance computing, we matched our converted IP numbers against the database. If an IP number fell within a specific IP_FROM and IP_TO range, the corresponding Country Name was retrieved.

Finally, we merged these recovered country names back into the main dataset, successfully filling in the gaps where the country was previously unknown.

Before	`> colSums(is.na(df))`
	`Date Notify URL IP Country WebServer Encoding Ransom DownTime Loss`
	`0 3673 3597 31323 32966 8707 904 0 0 0`

After	`> colSums(is.na(df))`
	`Date Notify URL IP Country WebServer Encoding Ransom`
	`0 3673 3597 31323 1579 8707 904 0`
	`DownTime Loss`
	`0 0`

1.3.6 Handle Missing Values (Removal)

After applying the imputation methods and the IP geolocation recovery, we performed a final inspection to check for any remaining missing values in the categorical columns. We generated a summary table to calculate the percentage of missing data for each variable., Figure 1.3.1.26.

	Column	Missing_Count	Percentage
Notify	Notify	3673	0.62%
URL	URL	3597	0.61%
Encoding	Encoding	904	0.15%
Date	Date	0	0%
IP	IP	0	0%
Country	Country	0	0%
WebServer	WebServer	0	0%
Ransom	Ransom	0	0%
DownTime	DownTime	0	0%
Loss	Loss	0	0%

Figure 1.3.1. 26

As shown in the table above, the Data Recovery step (Section 1.3.5) was highly successful, reducing the missing values for IP and Country to 0%. The remaining missing data was found only in the Notify (0.62%), URL (0.61%), and Encoding (0.15%) columns. Since these percentages are extremely low (less than 1% of the total dataset), removing these rows would not significantly impact the sample size or the integrity of our analysis.

Therefore, we proceeded to drop the rows containing missing values in these specific columns. Finally, we ran the colSums() function one last time to verify the dataset was completely clean, Figure 1.3.1.27.

```
> colSums(is.na(df))
     Date    Notify       URL        IP   Country WebServer  Encoding    Ransom  DownTime      Loss
        0         0         0         0         0         0         0         0         0         0
```

Figure 1.3.1. 27

1.3.7 Data Transformation – Scaling

To ensure our numerical data was suitable for analysis, we first identified the three key numerical variables: Ransom, DownTime, and Loss. Before applying any transformation, we visualized the distribution of these variables using histograms to understand their structure. Additionally, we calculated the skewness value for each variable using the skewness() function to quantify the asymmetry, Figure 1.3.1.28.

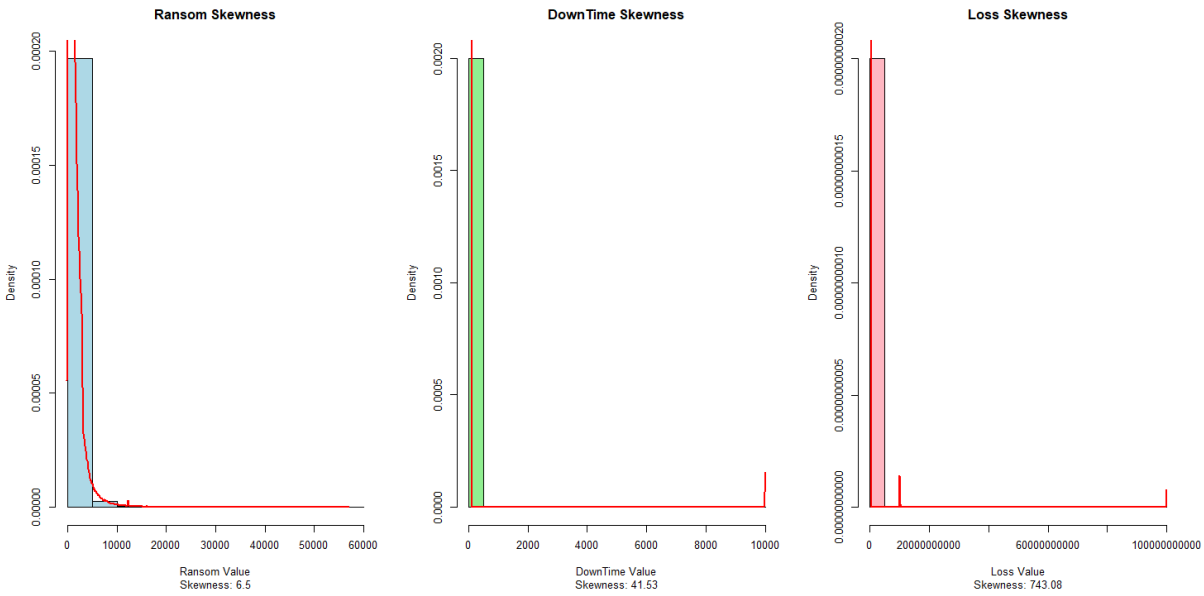


Figure 1.3.1. 28

As observed in the figure above, all three variables exhibited a highly right-skewed distribution (long tail to the right). For instance, the Loss variable had an extreme skewness value of +743.08. To address this and bring the data closer to a normal distribution, we decided to apply a Log Transformation.

However, we had to choose between the standard log() function and the log1p() function. Since our dataset contains zero values (e.g., cases where the ransom demanded was 0), using the standard log(x) function would result in mathematical errors (returning -Inf or NaN), as the logarithm of zero is undefined.

To solve this, we utilized the log1p() function, which calculates log(x + 1). By adding 1 to the value before taking the logarithm, log1p allows us to handle zero values safely while effectively normalizing the distribution, Figure 1.3.1.29.

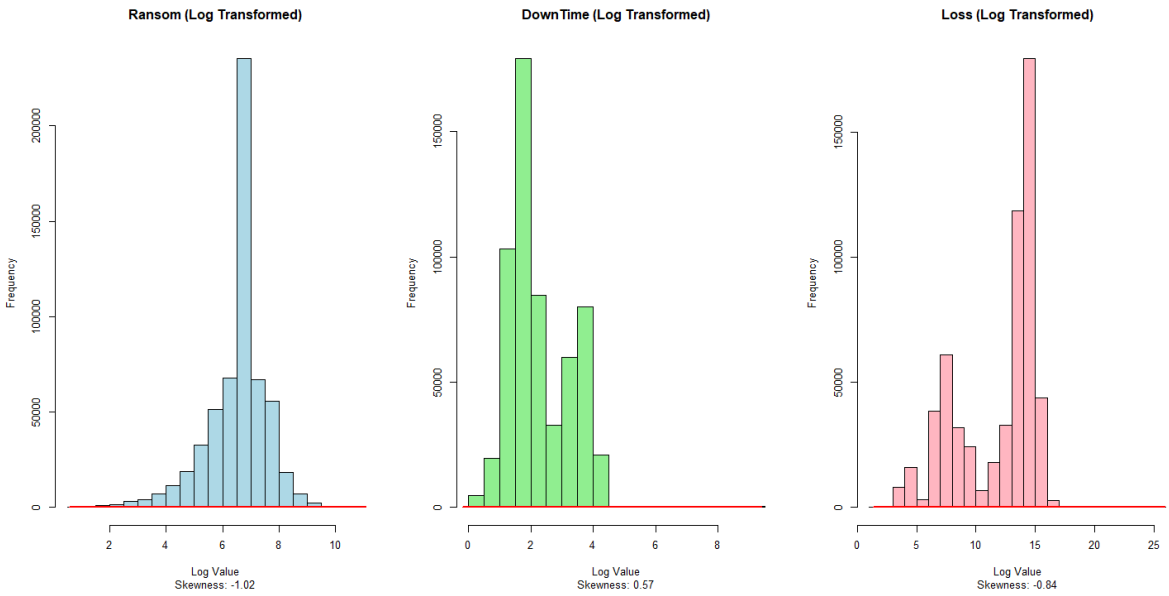


Figure 1.3.1. 29

Comparison Table

Feature	Skewness (Before)	Skewness (After)	Result
Ransom	+6.5	-1.02	Improved
DownTime	+41.53	+0.57	Improved
Loss	+743.08	-0.84	Improved

1.3.8 Outlier Detection and Handling

We inspected the numerical feature (Ransom, DownTime and Loss) using histogram and statistical summaries to identify anomalies. By using the “skewness” and “sapply” function, we developed a table named “skew_table” that stored the skewness value for the numerical feature inside the dataset, Figure 1.3.1.30.

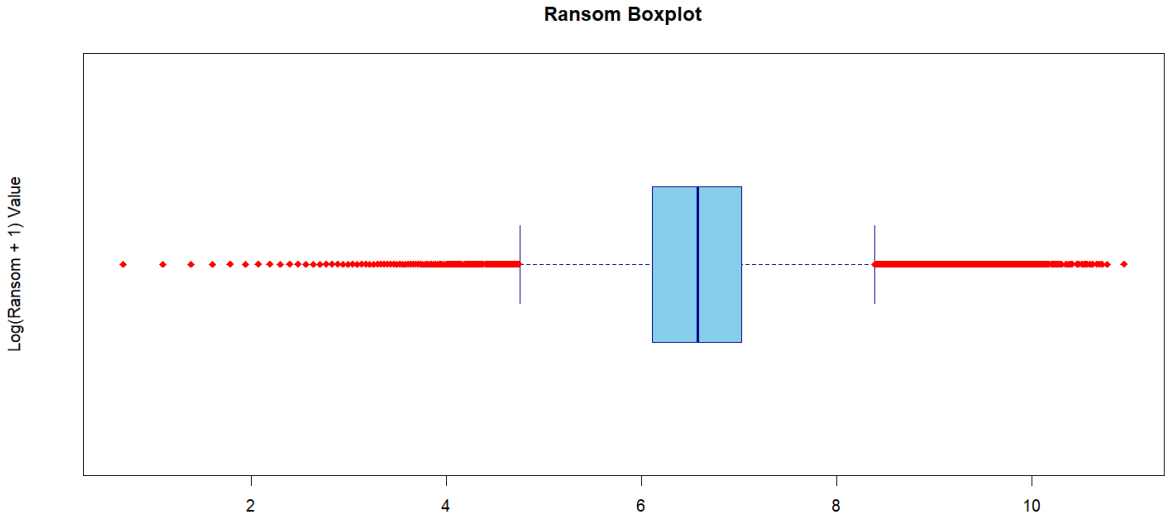
```
# Check feature skewness for each numerical feature
df_numeric <- df %>%
  select(where(is.numeric))

skew_values <- sapply(df_numeric, skewness, na.rm = TRUE)
skew_table <- data.frame(
  Feature = names(skew_values),
  Skewness = round(skew_values, 2),
  Interpretation = case_when(
    skew_values > 1 ~ "Highly Right Skewed",
    skew_values < -1 ~ "Highly Left Skewed",
    abs(skew_values) <= 0.5 ~ "Symmetrical (Normal-ish)",
    TRUE ~ "Moderately Skewed"
  )
)

skew_table <- skew_table %>%
  arrange(desc(Skewness))
```

```
> print(skew_table)
            Feature Skewness     Interpretation
DownTime DownTime     0.57  Moderately Skewed
Loss         Loss    -0.84  Moderately Skewed
Ransom     Ransom    -1.02 Highly Left Skewed
```

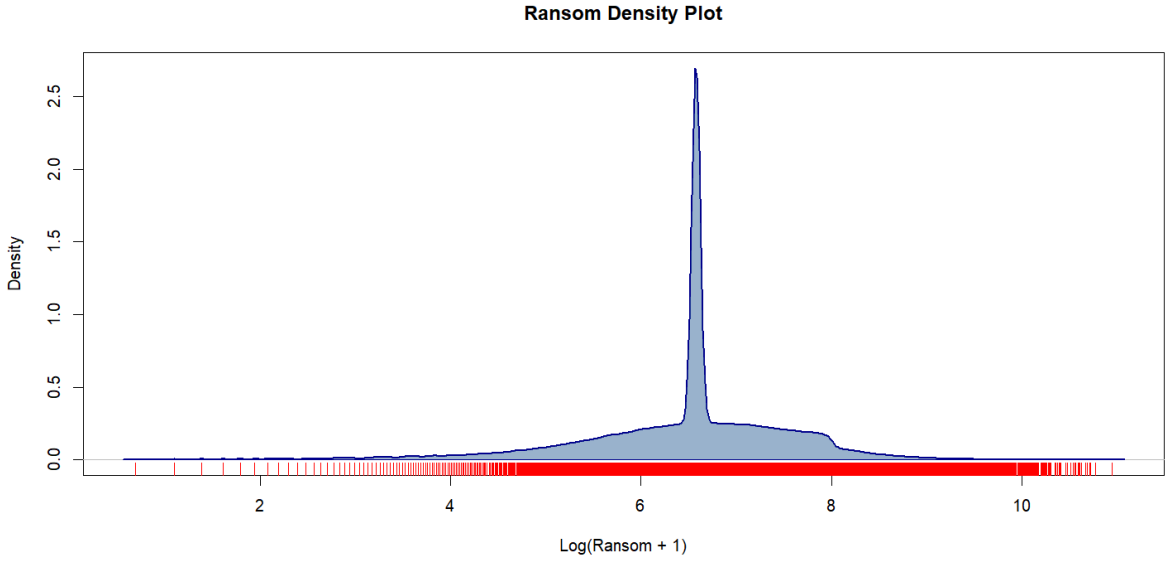
Figure 1.3.1. 30

Ransom

By checking the boxplot and density plot for the Ransom variable data point distribution, we can clearly confirmed that the there are a outliers inside the Ransom variable.



Box Plot



Density Plot

According to the visualization above, we can notice that there are many extreme outliers exist in the Ransom column, hence we decided to use the modified z-score to detect the outliers.

We also run a test on between the method used to detect the outliers, which is Interquartile Range (IQR) method and Modified Z-score method. There difference between the results

from the outlier detection from both detection method is the detected outliers number and the outliers value in the middle, Figure 1.3.1.31 to .32.

```
Method 1: IQR Method
> message(paste("Total Outlies detected: ", length(real_ransom_iqr)))
Total Outlies detected:  48924
> message(paste("Smaller outliers: ", head(real_ransom_iqr), "\n"))
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
> message(paste("Largest outliers: ", tail(real_ransom_iqr), "\n"))
Largest outliers:  44004
Largest outliers:  45141
Largest outliers:  45365
Largest outliers:  45435
Largest outliers:  47768
Largest outliers:  56781
```

Figure 1.3.1. 31

```
Method 2: Modified Z-score Method
> message(paste("Total Outlies detected: ", length(real_ransom_modz)))
Total Outlies detected:  6421
> message(paste("Smaller outliers: ", head(real_ransom_modz), "\n"))
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
Smaller outliers:  1
> message(paste("Largest outliers: ", tail(real_ransom_modz), "\n"))
Largest outliers:  44004
Largest outliers:  45141
Largest outliers:  45365
Largest outliers:  45435
Largest outliers:  47768
Largest outliers:  56781
```

Figure 1.3.1. 32

To determine whether this was a data entry error or a legitimate high-profile attack, we conducted external research into real-world cybersecurity statistics. According to a report by *Brian Atkinson (August 2024)*, the largest recorded ransom payment globally is approximately $75 million. Since the maximum value in our dataset is lower than this global record, it suggests that our data points are plausible and likely represent rare but severe real-world events.

Given that cyberattacks often involve extreme financial demands, using a standard outlier removal method (like IQR) would incorrectly delete valuable data. Therefore, we adopted the Modified Z-Score method, which is more robust for detecting outliers in non-normal distributions. We set a threshold of 3.5 to identify only the most extreme cases.

Instead of removing these outliers, we decided to flag them. We created a new column named Ransom_Outliers_Flag (assigned '1' for outliers and '0' for normal data). This approach allows us to retain these high-value attacks for analysis while still being able to distinguish them from typical ransomware incidents, Figure 1.3.1.33.

```
> df$Ransom_Outliers_Flag <- ifelse(abs(ransom_mod_z) > 3.5, 1, 0)
> cat("Final Count of Ransom Outliers flagged:", sum(df$Ransom_Outliers_Flag))
Final Count of Ransom Outliers flagged: 6421
```

Figure 1.3.1. 33

DownTime

Finally, we analysed the **Loss** variable. Similar to DownTime, we started by checking the distribution using a boxplot.Figure 1.3.1.34.

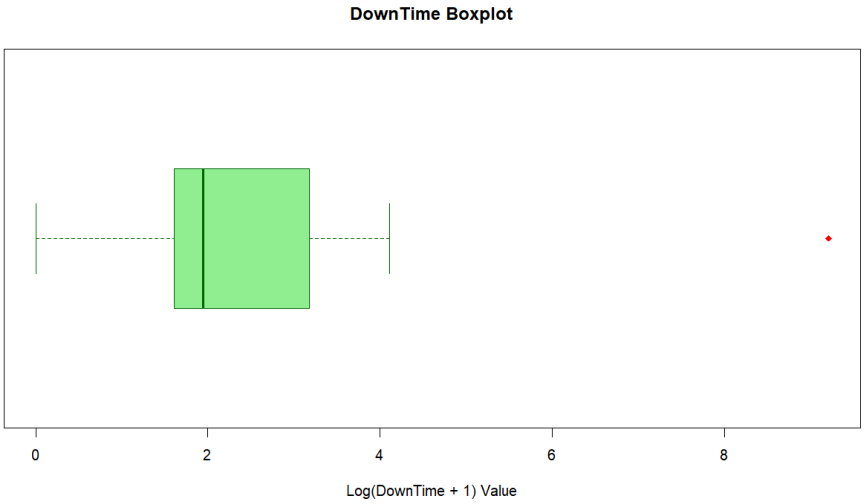


Figure 1.3.1. 34

The boxplot displayed several outliers beyond the upper whisker. To investigate further, we applied the IQR Method to isolate these values and inspected the actual numbers (converted back from log scale to real money), Figure 1.3.1.35.

```
> print(real_downtime_outliers)
  [1] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
 [22] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
 [43] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
 [64] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
 [85] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[106] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[127] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[148] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[169] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[190] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[211] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[232] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[253] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[274] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[295] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
[316] 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999 9999
```

Figure 1.3.1. 35

Upon closer inspection, we noticed a suspicious pattern: the outlier values were almost exclusively "perfectly round" integers (e.g., exactly 1,000,000 or 500,000). In real-world financial data, losses usually involve specific, uneven amounts (like 12,450.32). The presence

of these perfect integers strongly suggests that these records were estimates, dummy data, or rounded figures entered during a quick report, rather than actual calculated financial losses.

Since these "estimated" values could distort the accuracy of our specific financial analysis, we decided to remove these rows from the dataset.

After removing them, we plotted the boxplot one last time to confirm that the variable was clean, Figure 1.3.1.36.

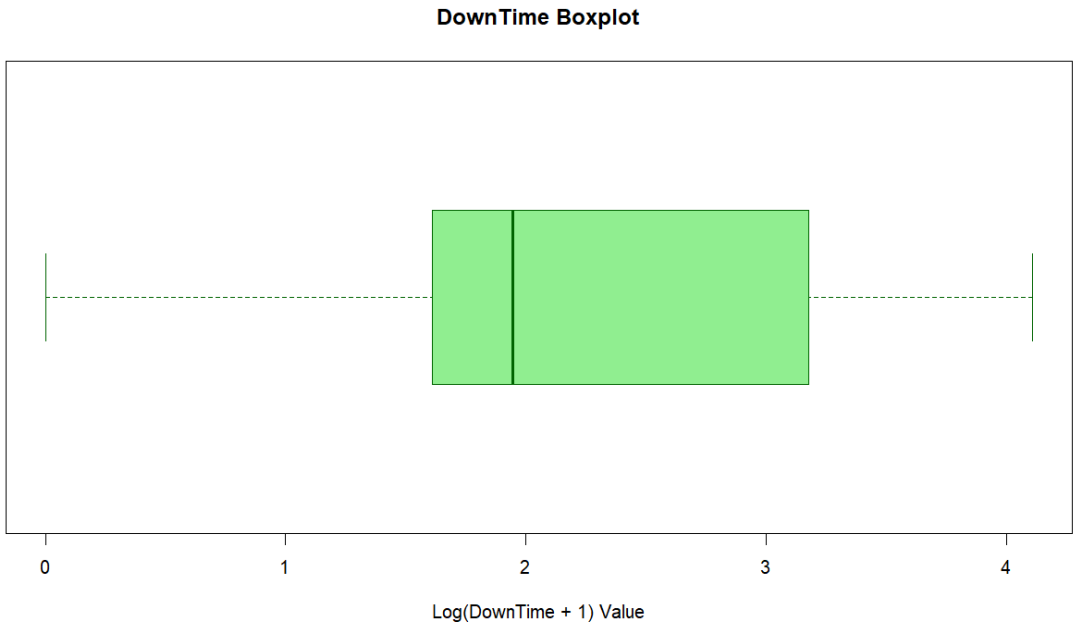


Figure 1.3.1. 36

Loss

We began by reviewing the summary statistics, Figure 1.3.1.37.

```
> summary(df$Loss)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.946   8.342  13.614  11.878  14.429  25.328
```

Figure 1.3.1. 37

As shown in the summary, there is a significant discrepancy between the 3rd Quartile (14.429) and the Maximum value (25.328). This large gap indicates the presence of extreme outliers. To visualize this distribution, we generated a boxplot, Figure 1.3.1.38.

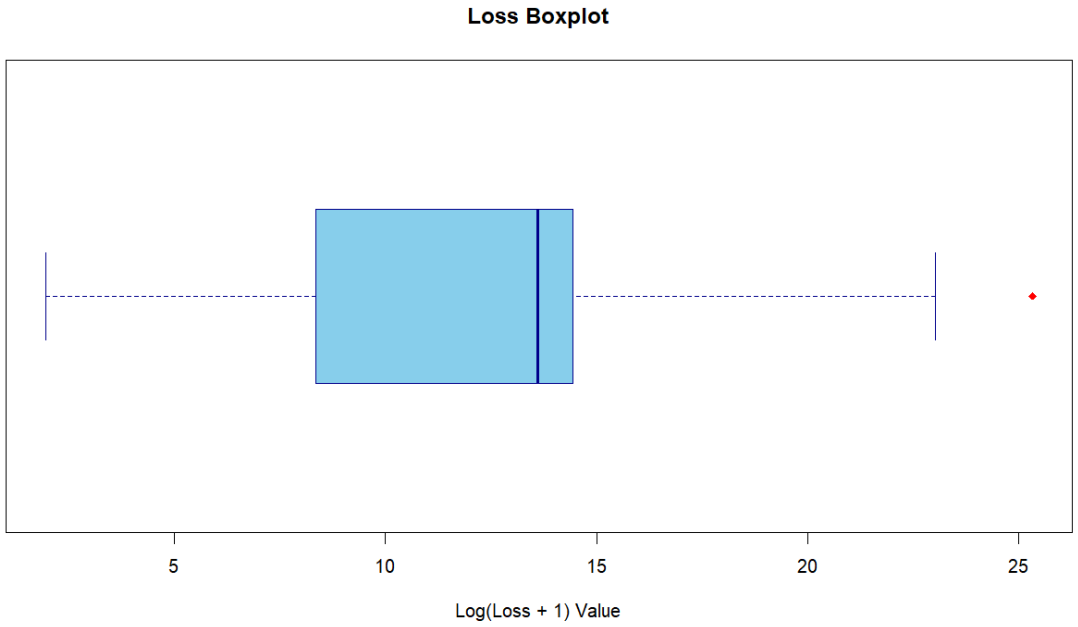


Figure 1.3.1. 38

The boxplot confirms the presence of outliers extending far beyond the upper whisker. To determine the nature of these values, we applied the **Interquartile Range (IQR)** method to isolate the outliers and converted them from their log-transformed state back to their original monetary values, Figure 1.3.1.39.

```
> print(real_loss_outliers)
[1] 100000000000
```

Figure 1.3.1. 39

Upon inspection, we observed that the outlier values were highly suspicious. The detected amounts included figures such as "9,999,999,900" and "10,000,000,000". In real-world financial contexts, loss calculations typically result in precise, uneven figures (e.g., $12,450.32) rather than such "perfectly round" integers.

We interpreted these values as system caps, default placeholders, or data entry errors rather than genuine financial figures. Including such artificial data would severely distort our analysis. Consequently, we decided to remove these rows from the dataset.

Following the removal, we plotted the boxplot again to verify the data quality. As seen below, the distribution is now consistent and free of these artificial anomalies, Figure 1.3.1.40.

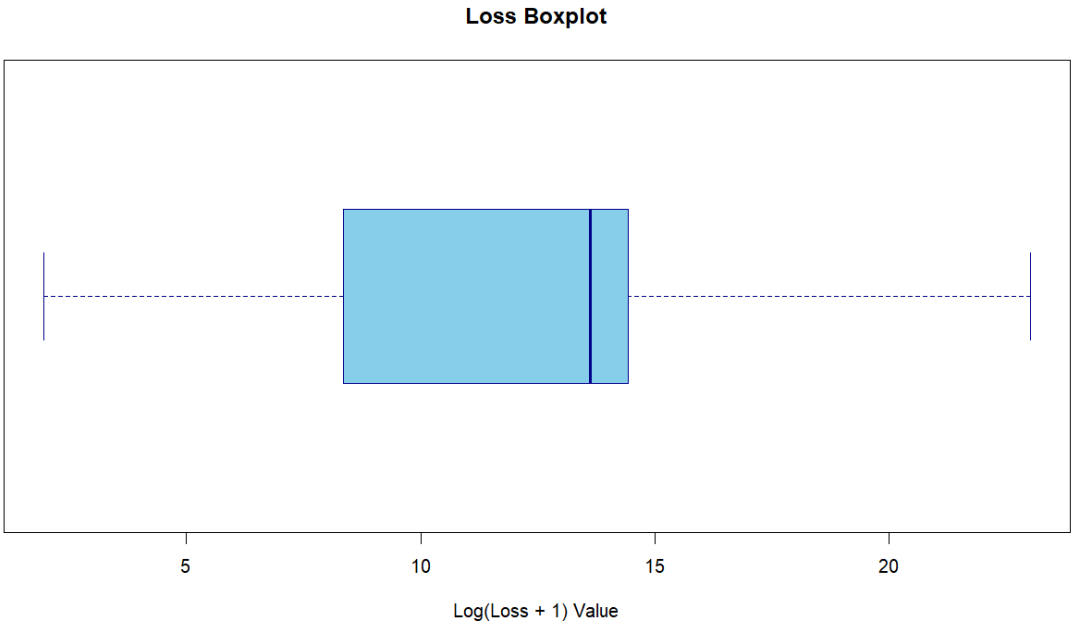


Figure 1.3.1. 40

1.3.9 Export Cleaned Dataset

Upon completing the outlier treatment and imputation processes, we successfully prepared a clean, consistent, and structured dataset ready for analysis. To facilitate the subsequent stages of this project, we exported the final dataframe to a CSV file named "HackingData_Cleaned.csv" using the "write.csv" function, Figure 1.3.1.41.

This step ensures that future analysis can proceed by simply loading this pre-processed file, thereby avoiding the computational cost of re-executing the entire data cleaning, recovery, and imputation pipeline.

```
########## Save cleaned dataset ##########
write.csv(df, "HackingData_Cleaned.csv", row.names = FALSE)
message("Clean Process Completed!")
```

Figure 1.3.1. 41

1.4 Assumptions

During data preparation, several assumptions were made to ensure the datasets was suitable for analysis. Firstly, it was assumed that all datasets represent the same information to allow the files to be combined into a single dataset without inconsistency. All records were assumed to be legitimate from actual cyber-attack events. Standardization of data type, encoding technique, and country names were assumed not to affect the original data. The date and time were assumed to be accurate. Each URL and IP address were assumed to be unique and useful in identifying geographic origin. Missing values were assumed to show that the data was unavailable. With the extreme skewness in the distribution of numerical variables like ransom, loss, and downtime, median imputation was assumed to represent real incidents are not outliers. Finally, it was assumed that the cleaned and combined dataset represents the cyber-attack incident correctly, and the data cleaning steps did not change the overall patterns.

2.0 Analysis

Leong Yuan Kang TP076118

2.1.1 Objective & Analysis Question

Objective:

To investigate the effect of system unavailability duration and temporal trends on the Ransom amount to assess if attack severity drives higher payments

Analysis Question:

- How does the average Ransom amount vary across different levels of system unavailability
- Has the average Ransom demand increase significantly over the years?

2.1.2 Data Preparation

Prior to conducting Exploratory Data Analysis (EDA) and Hypothesis Testing, the dataset underwent a rigorous transformation process using the tidyverse package in R to ensure data quality and analytical relevance.

First, a specialized dataframe named df_eda was constructed derived from the original df dataset. This process involved the generation of four additional variables designed to facilitate intuitive interpretation. To interpret the financial and operational metrics in their original units, Ransom_Raw and DownTime_Raw were created by applying the expm1() function to reverse the log-transformation of the original variables, Diagram 2.1.3.1. Additionally, a Year variable was extracted from the Date column to enable temporal analysis. A categorical variable, DownTime_Level, was also derived by binning

DownTime_Raw into three distinct severity categories: "Short (<1 Day)", "Medium (1-7 Days)", and "Long (>7 Days)", Figure 2.1.3.2.

```
# Data Preparation
df_eda <- df %>%
  select(Date, DownTime, Ransom) %>%
  filter(!is.na(Date), !is.na(DownTime), !is.na(Ransom)) %>%
  mutate(
    Ransom_Raw = expm1(Ransom),
    DownTime_Raw = expm1(DownTime),
    Year = year(as.Date(Date)),
    DownTime_Level = cut(DownTime_Raw,
                         breaks = c(-Inf, 1, 7, Inf),
                         labels = c("Short (<1 Day)", "Medium (1-7 Days)", "Long (>7 Days)"))
  )
```

Figure 2.1.3. 1

```
# Data Preparation
df_eda <- df %>%
  select(Date, DownTime, Ransom) %>%
  filter(!is.na(Date), !is.na(DownTime), !is.na(Ransom)) %>%
  mutate(
    Ransom_Raw = expm1(Ransom),
    DownTime_Raw = expm1(DownTime),
    Year = year(as.Date(Date)),
    DownTime_Level = cut(DownTime_Raw,
                         breaks = c(-Inf, 1, 7, Inf),
                         labels = c("Short (<1 Day)", "Medium (1-7 Days)", "Long (>7 Days)"))
  )
```

Figure 2.1.3. 2

Subsequently, two custom functions, num_summary and cat_summary, were defined to streamline the generation of descriptive statistics, Diagram 2.1.3.3 & 2.1.3.4.. These functions were designed to automatically calculate key metrics including Minimum, Maximum, Mode, Median Absolute Deviation (MAD), and Robust Coefficient of Variation (RCV) to ensuring consistent and comprehensive statistical reporting throughout the analysis.

```r
# Self defined function for statistical summary
cat_summary <- function(x) {
  freq_table <- table(x, useNA = "ifany")
  prop_table <- prop.table(freq_table)
  perc_table <- prop_table * 100
  mode_val <- names(freq_table)[which.max(freq_table)]

  #Combine into a Data Frame
  summary_df <- data.frame(
    Category = names(freq_table),
    Frequency = as.numeric(freq_table),
    Proportion = as.numeric(prop_table),
    Percentage = as.numeric(perc_table)
  )

  # Sort by Frequency
  summary_df <- summary_df[order(-summary_df$Frequency), ]

  return(summary_df)
}
```

Figure 2.1.3. 3

```r
num_summary <- function(x) {
  get_mode <- function(v) {
    uniqv <- unique(v[!is.na(v)])
    uniqv[which.max(tabulate(match(v, uniqv)))]
  }
  x_clean <- x[!is.na(x)]
  q <- quantile(x_clean, probs = c(0.25, 0.75))
  med <- median(x_clean)
  mad_val <- mad(x_clean, constant = 1)
  res <- c(
    Min    = min(x_clean),
    Max    = max(x_clean),
    Q1     = q[1],
    Q3     = q[2],
    IQR    = IQR(x_clean),
    Mean   = mean(x_clean),
    Median = med,
    Mode   = get_mode(x_clean),
    MAD    = mad_val,
    RCV    = mad_val / med
  )

  return(res)
}
```

Figure 2.1.3. 4

2.1.3 Data Analysis

To establish a baseline understanding of the attack severity, we analyzed the descriptive statistics for Ransom_Raw, DownTime_Raw, and the categorical DownTime_Level.

The statistical analysis of Ransom_Raw reveals a highly skewed financial landscape within the dataset, Figure 2.1.4.1. A significantdifference between the Mean and the Median show that while the typical ransom demand is relatively moderate, the average is still heavily skewed by the other small number of extreme outliers. Maximum value highlights the exist of "Big Game Hunting" strategy, where the threat actors focus on high-value organizations for larger ransom. Furthermore, high Robust Coefficient of Variation (RCV) represent instability in ransom pricing, demands are not standardized but are likely tailored to the specific victim's revenue or the perceived value of their compromised data.

```
> # Statistical Data
> message("Statistical Summary: Real Ransom Amount")
Statistical Summary: Real Ransom Amount
> print(num_summary(df_eda$Ransom_Raw))
          Min           Max        Q1.25%        Q3.75%           IQR          Mean        Median
    1.0000000 56781.0000000   447.0000000  1131.0000000   684.0000000  1042.8817819   717.0000000
         Mode           MAD           RCV
  717.0000000   315.0000000     0.4393305
```

Figure 2.1.4. 1

For the DownTime_Raw variable, the statistics provide insight for the operational resilience of victims, Figure 2.1.4.2. The Median is the reliable benchmark for the "standard" recovery period, representing the typical time an organization not operating due to an attack. The Interquartile Range (IQR) is particularly telling us that wider IQR suggests a significant gap in preparedness, where some of the organizations downtime is short due to robust backups, while others suffer prolonged outages. The Maximum downtime underscores the extreme operational risk by identifying cases of business interruption.

```
> message("Statistical Summary: Real DownTime (Days)")
Statistical Summary: Real DownTime (Days)
> print(num_summary(df_eda$DownTime_Raw))
     Min       Max    Q1.25%    Q3.75%       IQR      Mean    Median      Mode       MAD       RCV
 0.00000 60.00000  4.00000 22.00000 18.00000 14.26481  6.00000  4.00000  3.00000  0.50000
```

Figure 2.1.4. 2

The categorical analysis of DownTime_Level helps quantify the probability of different scenarios, Figure 2.1.4.3. By identifying the Mode (the most frequent category) allows us to determine the standard attack record whether most incidents are merely short-term nuisances or significant disruptions. Crucially, analyzing the percentage of attacks classified as "Long (>7 Days)" could provide us direct measure of systemic risk. If the percentage is big, it indicates that long downtime is not a rare exception but it is a common consequence of modern cyber-attacks, validated that the need for stronger disaster recovery strategies.

```
> message("Frequency of DownTime Severity")
Frequency of DownTime Severity
> print(cat_summary(df_eda$DownTime_Level))
           Category Frequency Proportion Percentage
2 Medium (1-7 Days)    281137 0.48855581  48.855581
3    Long (>7 Days)    270301 0.46972517  46.972517
1    Short (<1 Day)     24007 0.04171902   4.171902
```

Figure 2.1.4. 3

To address the first analysis question, "Does longer downtime lead to higher ransom demands?" a grouped summary dataframe, downtime_summary was created by running below code, Figure 2.1.4.4. This summary aggregated the un-logged Ransom_Raw data by the DownTime_Level categories defined in the preparation stage.

```
> # Grouped DownTime_Level Summary
> downtime_summary <- df_eda %>%
+   group_by(DownTime_Level) %>%
+   summarise(
+     Avg_Ransom_Real = mean(Ransom_Raw, na.rm = TRUE),
+     Median_Ransom_Real = median(Ransom_Raw, na.rm = TRUE),
+     Count = n()
+   )
> print(downtime_summary)
# A tibble: 3 × 4
  DownTime_Level    Avg_Ransom_Real Median_Ransom_Real  Count
  <fct>                       <dbl>              <dbl>  <int>
1 Short (<1 Day)              1104.               658.  24007
2 Medium (1-7 Days)           1113.               652. 281137
3 Long (>7 Days)               964.               717. 270301
```

Figure 2.1.4. 4

Given the high variance identified earlier, both the Mean and Median were calculated to provide a balanced view of central tendency. A grouped bar chart was generated to visualize these metrics side-by-side, Figure 2.1.4.5.

```
# Visualization for downtime_summary
downtime_long <- downtime_summary %>%
  pivot_longer(cols = c(Avg_Ransom_Real, Median_Ransom_Real),
               names_to = "Metric",
               values_to = "Value")

ggplot(downtime_long, aes(x = DownTime_Level, y = Value, fill = Metric)) +
  # "position_dodge" puts bars side-by-side
  geom_col(position = position_dodge(width = 0.9)) +
  # Add labels
  geom_text(aes(label = paste0("$", scales::comma(round(Value, 0)))),
            position = position_dodge(width = 0.9),
            vjust = -0.5,
            fontface = "bold",
            size = 3.5) +
  labs(title = "Mean vs Median Ransom Amount by Downtime Severity",
       x = "DownTime Level",
       y = "Ransom Amount in Thounsand($)") +
  theme_minimal() +
  scale_y_continuous(labels = scales::comma, limits = c(0, 1300)) +
  scale_fill_manual(values = c("Avg_Ransom_Real" = "#4e79a7", "Median_Ransom_Real" = "#f28e2b"),
                    labels = c("Mean", "Median"))
```

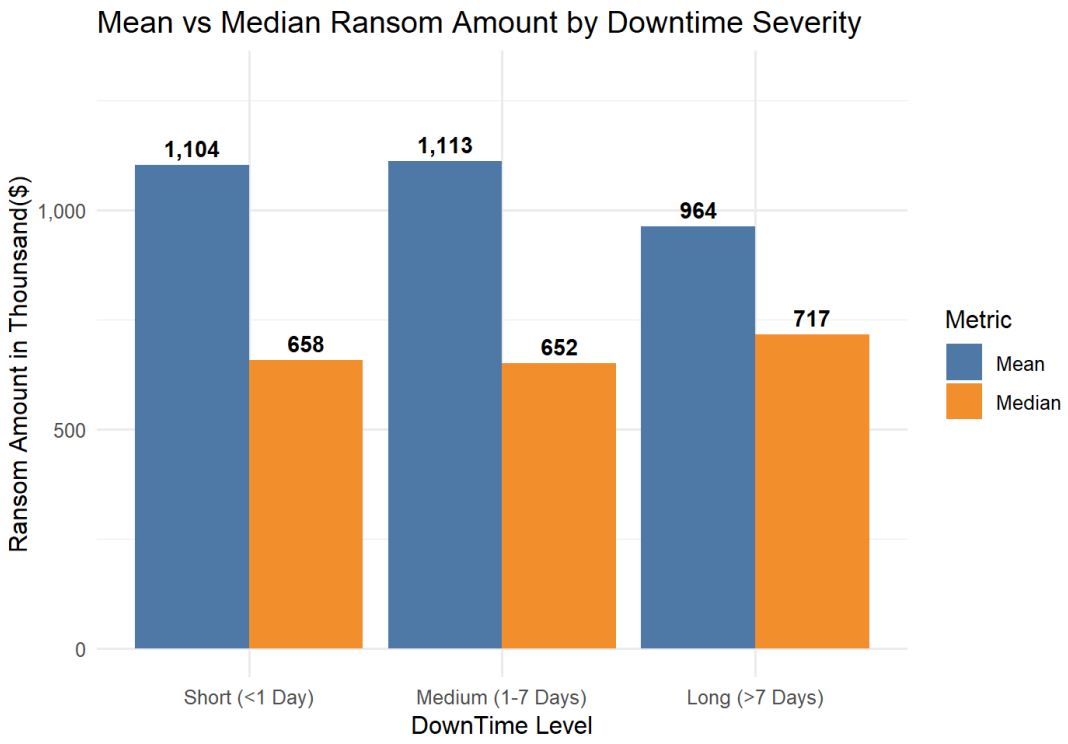


Figure 2.1.4. 5

The analysis reveals a nuanced relationship between operational downtime and financial demands, rather than a strictly linear correlation. Across all three downtime levels, the mean remains significantly higher than the median. Meanwhile, this confirms that “Big Game Hunting” occurs all severity levels, distorting the average price regardless of how long the system is down.

The medium downtime duration recorded the highest average ransom ($11.13 million). This suggests that attacks lasting roughly a week often involve high-value targets when the ransom demands are aggressive, driving up the mean.

Surprisingly, the long downtime category shows the lowest mean ($9.64 million) but the highest median ($7.17 million). The higher median indicates that the “typical” or baseline ransom for prolonged outages is indeed higher than short outages (supporting the hypothesis for the majority of cases). However, the lower mean suggests there are fewer massive multi-million dollar in this group compared to the medium downtime category.

The data suggests that while prolonged system unavailability (more than 7 days) is associated with a higher baseline (median) cost, the most astronomically expensive attacks tend to cluster in the medium downtime range.

To address the second analysis question, a temporal analysis was conducted to track the evolution of financial severity.

A summary dataframe, yearly_trend was derived by grouping the dataset by Year, Figure 2.1.4.6. For each annual period, two key metrics were calculated, median Ransom (Median_Ransom) that used to measure median financial cost of attacks, and the total attack count (Total_Attacks) to contextualize the volume of incidents. The data was sorted chronologically to ensuring a continuous time-series analysis. But somehow we noticed that if we using median for analysis, no much valuable information could be figure out due to majority of the ransom amount is extreme value. Hence, I decided to change the median to mean to explore more information. A line chart with distinct data point was generated to visualize the trajectory of ransom demands over time, Figure 2.1.4.7.

```
> # Analysis Question 2
> yearly_trend <- df_eda %>%
+   group_by(Year) %>%
+   summarise(
+     Median_Ransom = median(Ransom_Raw, na.rm = TRUE),
+     Total_Attacks = n()
+   ) %>%
+   arrange(Year)
> print(yearly_trend, n = 25)
# A tibble: 23 × 3
    Year Median_Ransom Total_Attacks
   <int>         <dbl>         <int>
 1  1998          717.            48
 2  1999          717.           853
 3  2000          717.          3905
 4  2001          717.         18537
 5  2002          717.         23809
 6  2003          717.         13865
 7  2005          717.           998
 8  2006          717.          1008
 9  2007          717.          1030
10  2008          717.           171
11  2009          717.         64784
12  2010          717.          1433
13  2011          717.          1008
14  2012          717.          1070
15  2013          717.         59352
16  2014          717.         12485
17  2015          717.           136
18  2020          654.         63820
19  2021          646.         63528
20  2022          649.         63454
21  2023          649.         63420
22  2024          654.         63589
23  2025          654.         53142
```

Median

```
> # Analysis Question 2
> yearly_trend <- df_eda %>%
+   group_by(Year) %>%
+   summarise(
+     mean_Ransom = mean(Ransom_Raw, na.rm = TRUE),
+     Total_Attacks = n()
+   ) %>%
+   arrange(Year)
> print(yearly_trend, n = 25)
# A tibble: 23 × 3
    Year mean_Ransom Total_Attacks
   <int>       <dbl>         <int>
 1  1998       1026.            48
 2  1999        922.           853
 3  2000        916.          3905
 4  2001        924.         18537
 5  2002        921.         23809
 6  2003        926.         13865
 7  2005        951.           998
 8  2006        907.          1008
 9  2007        915.          1030
10  2008        947.           171
11  2009        919.         64784
12  2010        924.          1433
13  2011        952.          1008
14  2012        926.          1070
15  2013        919.         59352
16  2014        917.         12485
17  2015        819.           136
18  2020       1115.         63820
19  2021       1112.         63528
20  2022       1109.         63454
21  2023       1109.         63420
22  2024       1108.         63589
23  2025       1109.         53142
```

Mean

Figure 2.1.4. 6

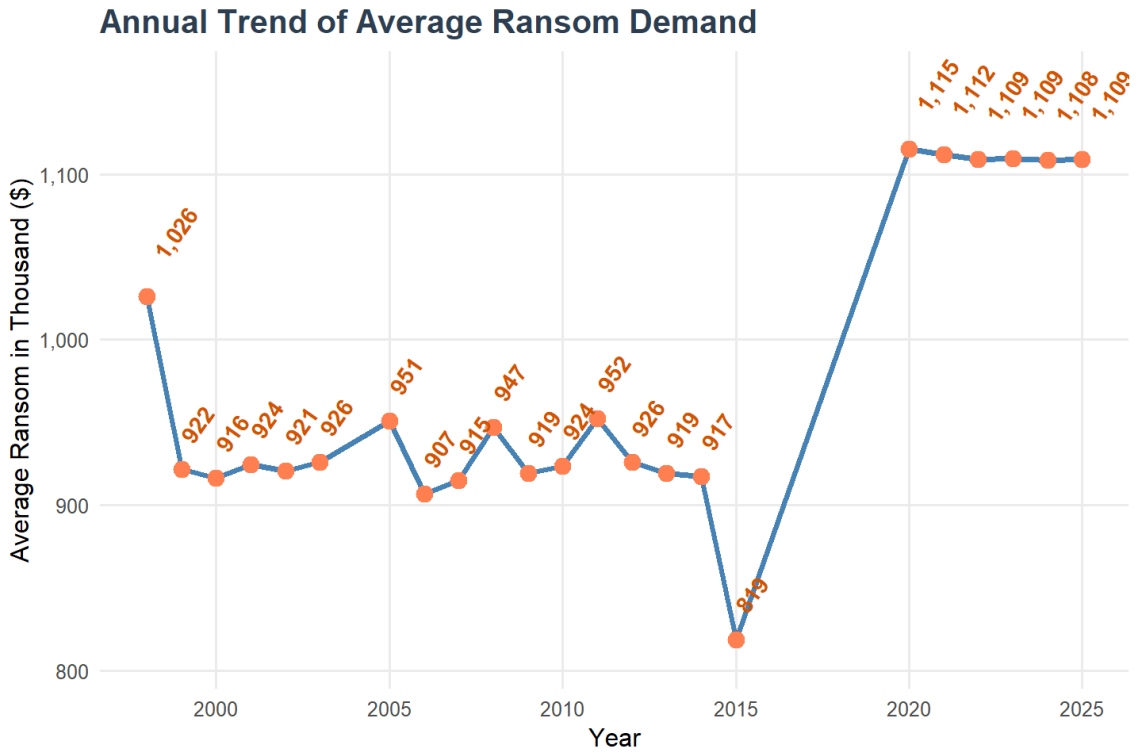


Figure 2.1.4. 7

The analysis of yearly trend reveals a striking shift in the economics of cyber attacks, characterizing two distinct eras. The “Classic” Era (1999 to 2014). For the nearly 15 years, the average ransom demand remained remarkably stable, hovering consistently between $9.07 million to $9.52 million. This stability reflects a period where ransomware attacks were largely automated and opportunistic, targeting individuals or small business with standardized and lower demands.

A notable anomaly occurred in 2015, where the average ransom dropped to its lowest records point at $8.19 million. However, this statistical dip contradicts the chaotic reality of that year. As noted by Business Insider, 2015 left as if “every month brought a major new cyberattack” ranging from the 37 million users affected in the Ashley Madison to the unprecedented breach of the US deferral agency (OPM) in charge of background checks. This discrepancy suggests that while the average ransom demand for common attacks was low, 2015 was actually a pivotal year where hackers shifted focus towards massive, high-profile data breaches, setting the stage for the more aggressive extortion tactics seen later.

The most critical finding is the sudden and drastic surge in costs starting in 2020. The average ransom jumped to **$11.15 million** and has remained elevated (above $11 million) through 2025. This sharp increase aligns with the global rise of "Big Game Hunting," where sophisticated ransomware cartels began selectively targeting large enterprises with multi-million dollar demands. This escalation is further corroborated by industry data; a research study by Deep Instinct reported that ransomware attacks increased by **435%** in 2020 compared to the previous year, with hundreds of millions of cyber-attacks attempted daily. This triple-digit spike in activity confirms that the financial surge observed in our data was not an isolated anomaly but part of a broader intensification of cyber warfare.

2.1.4 Hypothesis and Formulation and Testing

Before selecting the primary statistical test for our research objective, it is essential to determine if the numerical data follows a normal distribution.

Hypothesis formulation

H0: There is no significant difference in the median Ransom amount across different downtime severity levels (Short, Medium, Long).

H1: There is a **significant difference** in the median Ransom amount across different downtime severity levels.

Significance Level

0.05, that's mean we require a 95% confidence level attackers financial demands.

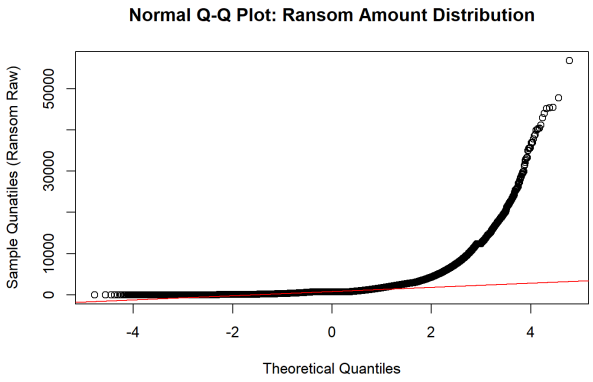
Test Results and Decision

The test yielded a test statistics of a p-value < 2.2e-16, Figure 2.1.5.1, therefore we compare the p-value to the significance level 0.05. As a result, the Ransom data is not normally distributed. Therefore, we are required to perform the test by using the Kruskal-Wallis Test.

```
> # Normality Check
> set.seed(123) # For reproducibility
> shapiro_results <- shapiro.test(sample(df_eda$Ransom_Raw, 5000))
> print(shapiro_results)

        Shapiro-Wilk normality test

data:  sample(df_eda$Ransom_Raw, 5000)
W = 0.53271, p-value < 0.00000000000000022
```

Figure 2.1.5. 1

```
> # QQ Plot for visual check
> qqnorm(df_eda$Ransom_Raw,
+        main = "Normal Q-Q Plot: Ransom Amount Distribution",
+        xlab = "Theoretical Quantiles",
+        ylab = "Sample Qunatiles (Ransom Raw)")
> qqline(df_eda$Ransom_Raw, col = "red")
```



The Kruskal-Wallis test produced a chi-squared value of 6035 with a p-value < 2.2e-16 ($p < 0.05$). To pinpoint exactly where the differences lie, we conducted a Pairwise Wilcoxon Test with a Bonferroni adjustment. The results revealed a fascinating pattern:

- The comparison between Short and Medium downtime yielded a p-value of 1.00, indicating no statistical difference.
- The comparison between Long downtime and all other groups yielded p-values < 0.05, indicating a highly significant difference.

```
> # Kruskal-Wallis Test
> kruskal_result <- kruskal.test(Ransom_Raw ~ DownTime_Level, data = df_eda)
> print(kruskal_result)

        Kruskal-Wallis rank sum test

data:  Ransom_Raw by DownTime_Level
Kruskal-Wallis chi-squared = 6035, df = 2, p-value < 0.00000000000000022

> pairwise.wilcox.test(df_eda$Ransom_Raw, df_eda$DownTime_Level, p.adjust.method = "bonferron
i")

        Pairwise comparisons using Wilcoxon rank sum test with continuity correction

data:  df_eda$Ransom_Raw and df_eda$DownTime_Level

                  Short (<1 Day)       Medium (1-7 Days)
Medium (1-7 Days) 1                    -
Long (>7 Days)    <0.0000000000000002 <0.0000000000000002

P value adjustment method: bonferroni
```

Since the p-value is significantly lower than 0.05, we reject Null Hypothesis (H0). We conclude that there is a statistically significant relationship between downtime duration and ransom demands. Specifically, while minor to moderate disruption result in similar ransom scale, a “tipping ppint” occurs once downtime exceed one week. At this stage, the attack severity triggers a significant shift in financial demand, confirming that prolonged system unavailability is a primary driver of higher ransom payments

Lew Li Jun TP087450

2.2.1 Objective & Analysis Question

Objective:

The purpose of this analysis is to examine the geographical disparities in cyber-attacks to understand if the hosting Country significantly influences the Ransom magnitude.

The objective is considered through two research questions.

Analysis Questions:

- Which Country records the highest average Ransom amounts?

- How do ransom demands vary across servers hosted in different regions?

2.2.2 Data Preparation

Before analysis, the dataset was cleaned to ensure accurate mapping of countries and regions. This helps to prevent errors in statistical analysis.

```
# Data Preparation
df_clean <- df %>%
  filter(!is.na(Country), !is.na(Ransom), Ransom > 0)

df_clean$Country[df_clean$Country %in% c("Unknown", "Others")] <- NA

df_clean$Country[df_clean$Country %in% c("Cocos (Keeling) Islands", "Heard & McDonald Islands",
                                         "South Georgia & South Sandwich Islands", "St. Helena")] <- NA

df_clean <- df_clean %>%
  filter(!is.na(Country))

df_clean$Region <- countrycode(df_clean$Country,
                               origin = "country.name",
                               destination = "region")

df_clean <- df_clean %>%
  filter(!is.na(Region))

sum(is.na(df_clean$Country))
unique(df_clean$Country)
glimpse(df_clean)

table(df_clean$Region)
```

Figure 2.2.3.1

First of all, rows with missing values in Country and Region were removed. Ransom values less than or equal to zero were filtered as they are not meaningful for analysis.

Then, non-specific entries like “Unknown” and “Others” were marked as NA because they do not correspond to real countries. Countries that could not be classified were excluded because they are too few and does not reliably map to a region. After that, all rows with NA countries were removed from the dataset.

By using the countrycode package, each country was mapped to its corresponding geographical region. Rows with missing values after mapping were removed to ensure all rows have valid regions for analysis. Thus, we can guarantee consistent and maintain meaningful regional comparison.

Finally, unique country names and region frequency tables were checked to verify the dataset was fully cleaned. Now the dataset is ready for analysis.

2.2.3 Data Analysis

Analysis Question 1

To determine which countries experienced the highest ransom demands, the dataset was first grouped by country. Since the ransom values had been log-transformed during the data preparation, the exponential transformation (expm1) was applied to return the values to original ransom values. The mean ransom payment was calculated for each country so that the analysis reflects the true value of ransom. The countries are sorted from highest average ransom to lowest. The code used to perform this analysis is shown in Figure 2.2.4.1 and the resulting table of average ransom is shown in Figure 2.2.4.2.

```
df_clean %>%
  group_by(Country) %>%
  summarise(avg_ransom = mean(expm1(Ransom), na.rm = TRUE)) %>%
  arrange(desc(avg_ransom))
```

Figure 2.2.4.1

	Country	avg_ransom
	<chr>	<dbl>
1	Afghanistan	2812.
2	St. Kitts & Nevis	2506.
3	U.S. Virgin Islands	2395
4	Suriname	1708.
5	Benin	1693.
6	Bahamas	1314.
7	Djibouti	1288.
8	New Caledonia	1257.
9	Qatar	1224.
10	Zambia	1223.

i 187 more rows

Figure 2.2.4.2

The top 10 countries with the highest average ransom values were selected for visualization. A bar chart was generated to reveals clear difference of average ransom values among these countries as shown in Figure 2.2.4.4. The country with the highest average ransom values shows that the country most affected by large-scale of ransomware attacks.

```r
highest_avg_ransom_country <- df_clean %>%
  group_by(Country) %>%
  summarise(avg_ransom = mean(expm1(Ransom), na.rm = TRUE)) %>%
  arrange(desc(avg_ransom)) %>%
  head(10)

ggplot(highest_avg_ransom_country, aes(x = reorder(Country, avg_ransom), y = avg_ransom, fill = avg_ransom)) +
  geom_col() +
  geom_text(aes(label = round(avg_ransom, 0)), vjust = 0.5, hjust = -0.2) +
  coord_flip() +
  labs(
    title = "Highest Average Ransom Countries",
    x = "Country",
    y = "Average Ransom(thousand)"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", size = 16, hjust = 0.5))
```

Figure 2.2.4.3

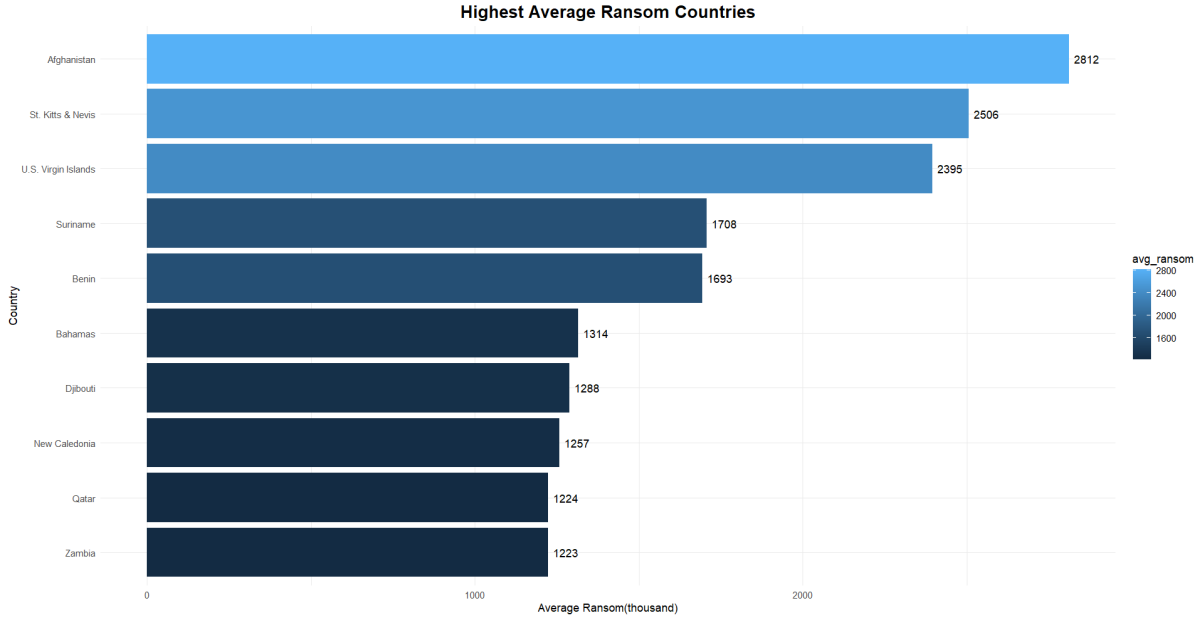


Figure 2.2.4.4

A bar chart was chosen for this analysis because it provides a clear visual comparison of average ransom with the countries. Average ransom amounts are shown in thousand units for readability. Since it focuses on the top 10 highest average ransom countries only, we can easily identify which country has the highest ransom.

The mean was used because the analysis focuses on average ransom values across countries. Although the mean is sensitive to extreme values, the ransom data were log-transformed to reduce the effect of outliers. Therefore, the mean provides a suitable measure for comparing average ransom values between countries.

Analysis Question 2

To identify which regions typically face higher ransom demands, the data was grouped by region. We calculated the summary statistics, including count, median, median absolute deviation(MAD), mean, and standard deviation(SD). The summary table is shown in Figure 2.2.4.6. The data was then sorted from highest to lowest median ransom for easier analysis.

```
df_clean %>%
  group_by(Region) %>%
  summarise(
    count = n(),
    median_ransom = median(Ransom, na.rm = TRUE),
    mad_ransom = mad(Ransom, na.rm = TRUE),
    mean_ransom = mean(Ransom, na.rm = TRUE),
    sd_ransom = sd(Ransom, na.rm = TRUE)
  ) %>%
  arrange(desc(median_ransom))
```

Figure 2.2.4.5

	Region	count	median_ransom	mad_ransom	mean_ransom	sd_ransom
	<fct>	<int>	<dbl>	<dbl>	<dbl>	<dbl>
1	East Asi…	169780	6.58	0.983	6.41	1.19
2	Europe &…	174889	6.58	0.675	6.49	1.05
3	Latin Am…	52950	6.58	0.770	6.47	1.09
4	Middle E…	3425	6.58	0	6.70	0.450
5	North Am…	101769	6.58	0.0784	6.58	0.858
6	Sub-Saha…	2460	6.58	0	6.72	0.464
7	South As…	38861	6.54	1.06	6.36	1.24

Figure 2.2.4.6

These statistics provide a clear overview of the typical ransom demands and their variability across regions. For example, **South Asia** shows the lowest median. MAD values indicate that **East Asia & Pacific** has relatively high variability compared to other regions.

```r
median_mad <- df_clean %>%
  group_by(Region) %>%
  summarise(
    median_ransom = median(Ransom, na.rm = TRUE),
    mad_ransom = mad(Ransom, na.rm = TRUE)
  )

ggplot(df_clean, aes(x=Region, y=Ransom, fill = Region)) +
  geom_boxplot(alpha = 0.7, color = "black") +
  stat_summary(fun = mean, geom = "point", shape = 20, color = "red", size = 3) +
  scale_fill_brewer(palette = "Set2") +
  theme_minimal() +
  geom_text(data = median_mad,
            aes(x = Region, y = median_ransom + 0.5,
                label = paste0("Med: ", round(median_ransom,2),
                               "\nMAD: ", round(mad_ransom,2))),
            color = "black", size = 3.5) +
  labs(title="Ransom Amounts by Region",
       x="Region",
       y="Ransom Amount") +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.title = element_text(face = "bold")
  )
```

Figure 2.2.4.7

A boxplot was created to compare ransom demands across regions as shown in Figure 2.2.4.7. A boxplot was chosen because it effectively displays the median, spread, and outliers for each region, making it easy to compare regional differences in ransom demands visually. The plot shows the median, quartiles, outliers, and mean (indicated by red points) for each region.

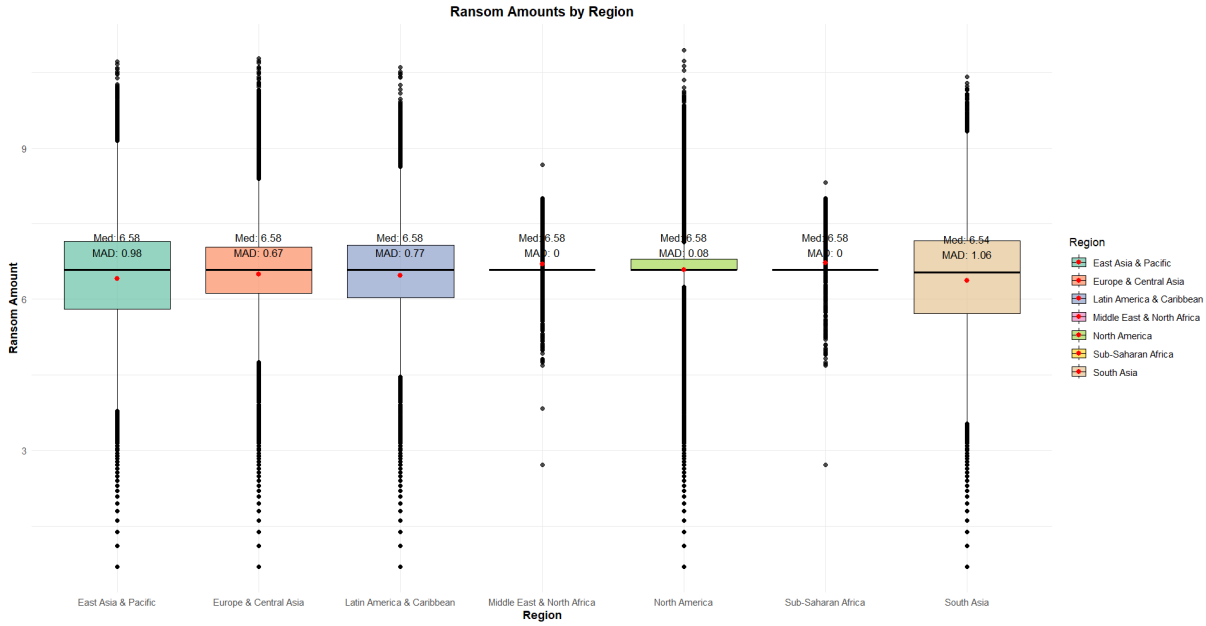


Figure 2.2.4.8

The boxplot reveals several extreme ransom values (outliers) in most regions, particularly in East Asia & Pacific and North America. These outliers indicate a few cases of exceptionally

high ransom payments, which could be due to large-scale cyber-attacks. While these extreme values increase the spread and affect the mean, they have less influence on the median, which provides a more robust measure of central tendency. Highlighting these outliers helps to understand the variability and potential risk in different regions. Some regions, such as Sub-Saharan Africa, have smaller sample sizes, which may limit the reliability of the estimates.

From the boxplot shown in Figure 2.2.4.8, it is evident that ransom amounts differ notably between regions. **East Asia & Pacific** and **Sub-Saharan Africa** exhibit higher median values and a wider spread, while **South Asia** has lower values. Outliers are present in most regions, highlighting some extremely high ransom cases. This visual comparison helps identify which regions generally experience higher or lower ransom demands.

2.2.4 Hypothesis Formulation and Testing

Hypothesis formulation

H0: The average ransom is the **same** across all regions.

H1: **At least one** region has a **different** average ransom across all regions.

Significance Level

The significance level is set at 0.05, which state that there is a 5% risk of rejecting the null hypothesis when it is actually true (Type I error).

Test Results and Decision

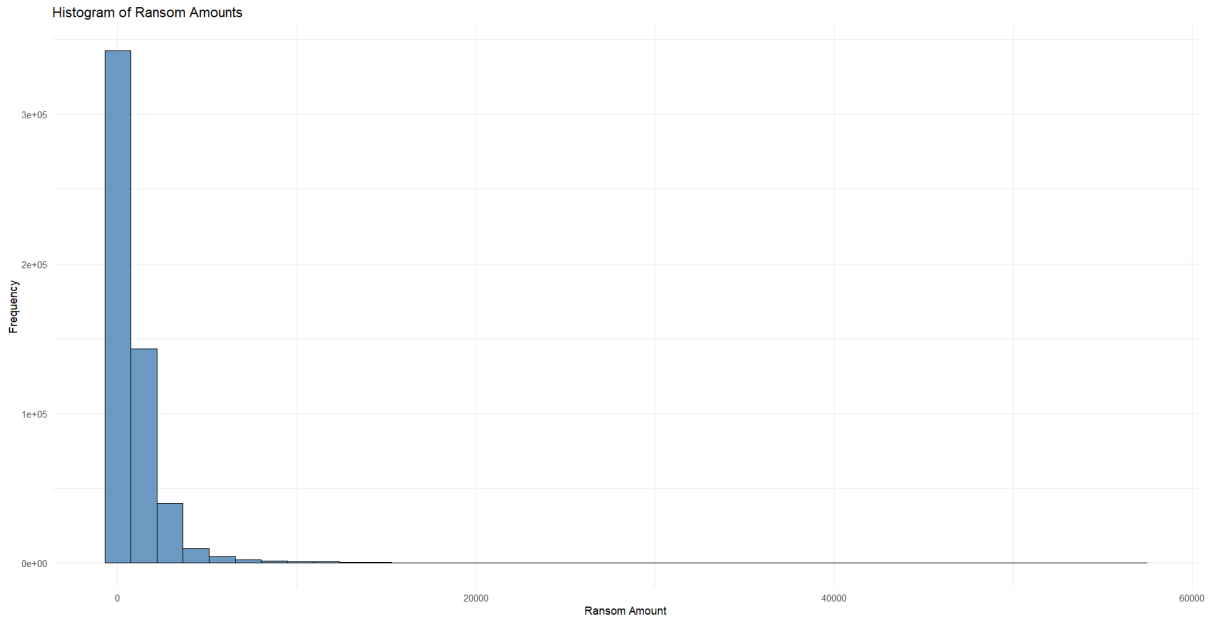


Figure 2.2.5.1

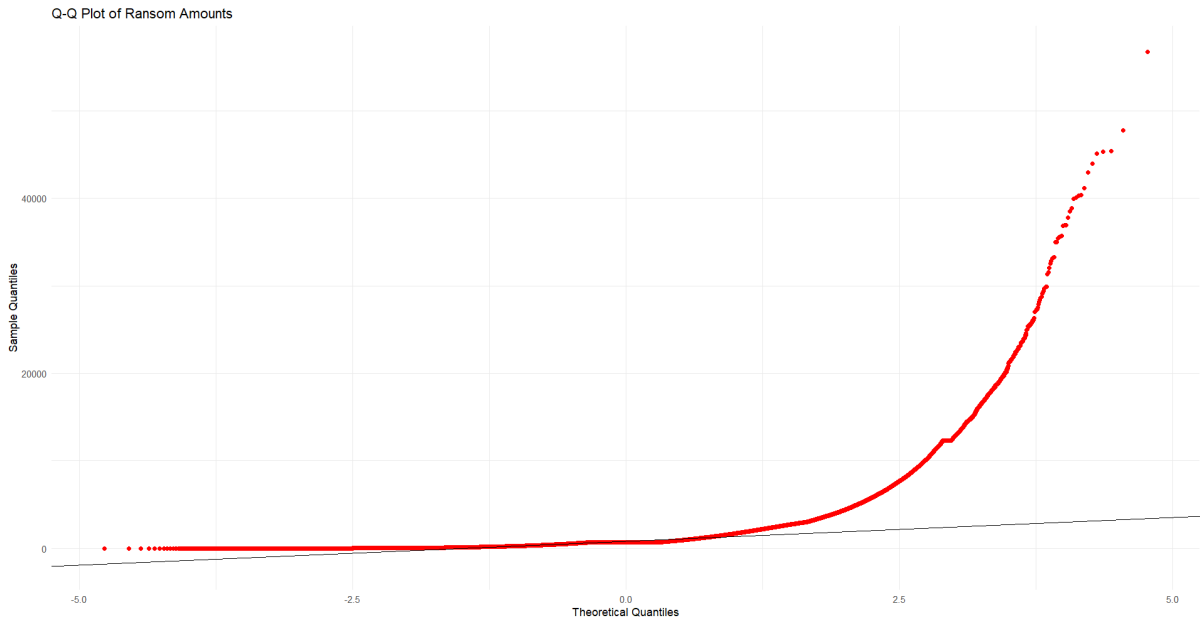


Figure 2.2.5.2

Since the data size is large, normality tests were not performed. Histograms and Q-Q plots were used, which indicated a large skewness in the data with non-normal distributions. Hence, a non-parametric test, Kruskal-Wallis, was preferred.

```
> kruskal.test(Ransom ~ Region, data = df_clean)

        Kruskal-Wallis rank sum test

data:  Ransom by Region
Kruskal-Wallis chi-squared = 1713, df = 6, p-value
< 2.2e-16
```

Figure 2.2.5.3

A Kruskal-Wallis rank sum test was selected to examine whether the ransom demands differ across the seven regions. This non-parametric test was selected because the ransom data is skewed and may not meet the normality assumption required for ANOVA. The ransom amount is a numerical variable, and the regions form a categorical variable with seven independent groups. The Kruskal-Wallis test is suitable for comparing one numerical variable across two or more independent groups.

The p-value < 2.2e-16 state that it is smaller than 0.05. Therefore, we reject the null hypothesis. This show that the ransom demands **differ significantly** across regions.

Carrie Yap Liu Qing TP075883

2.3.1 Objective & Analysis Question

Objective:

To evaluate the relationship between WebServer and the Ransom amount to identify specific vulnerabilities lead to higher financial demands

Analysis Questions:

- Do victims using outdated WebServer versions pay significantly higher Ransom compared to those using modern ones?

- Is there a significant variance in Ransom amounts across different WebServer types?

2.3.2 Data Preparation

This section describes the additional data preparation steps taken to support the analysis of the relationship between WebServer and Ransom amounts.

```r
# Extract major version safely
extract_major_version <- function(ws) {
  suppressWarnings(as.numeric(str_extract(ws, "(?<=/)[0-9]+(\\.[0-9]+)?")))
}

# Clean and standardise web server data
clean_server_data <- function(df) {
  df %>%
    select(WebServer, Ransom) %>%
    filter(!is.na(WebServer), !is.na(Ransom)) %>%
    mutate(
      Ransom = as.numeric(Ransom),

      WebServer_Type_Raw = word(WebServer, 1, sep = "[/\\s]"),
      Major_Version = extract_major_version(WebServer),
```

Figure 2.3.2.1

Raw dataset is prepared first so that it can be analyzed in a reliable way. The information of web server is commonly stored as unstructured text which cannot be directly used in statistical analysis like Apache/2.4.54 or nginx/1.16.1. The *extract_major_version* function is used to extract the numeric version from these server strings, while the cleaning steps keep only relevant columns, drop missing values and ransom amounts are stored as numeric values. These measures are important because statistical analysis requires consistent and complete data. Without proper cleaning, the results could be inaccurate or biased.

```
    Server_Status = case_when(
      str_detect(WebServer, regex("^iis", ignore_case = TRUE)) & Major_Version < 8   ~ "Outdated",
      str_detect(WebServer, regex("^iis", ignore_case = TRUE)) & Major_Version >= 8  ~ "Modern",

      str_detect(WebServer, regex("^apache", ignore_case = TRUE)) & Major_Version < 2.4  ~ "Outdated",
      str_detect(WebServer, regex("^apache", ignore_case = TRUE)) & Major_Version >= 2.4 ~ "Modern",

      str_detect(WebServer, regex("^nginx", ignore_case = TRUE)) & Major_Version < 1.18  ~ "Outdated",
      str_detect(WebServer, regex("^nginx", ignore_case = TRUE)) & Major_Version >= 1.18 ~ "Modern",

      str_detect(WebServer, regex("litespeed|openresty", ignore_case = TRUE)) ~ "Modern",
      TRUE ~ "Unknown"
    ),

    Clean_Type = case_when(
      str_detect(WebServer_Type_Raw, regex("^iis", ignore_case = TRUE)) ~ "IIS",
      str_detect(WebServer_Type_Raw, regex("^apache", ignore_case = TRUE)) ~ "Apache",
      str_detect(WebServer_Type_Raw, regex("^nginx", ignore_case = TRUE)) ~ "Nginx",
      str_detect(WebServer_Type_Raw, regex("^litespeed", ignore_case = TRUE)) ~ "LiteSpeed",
      str_detect(WebServer_Type_Raw, regex("^openresty", ignore_case = TRUE)) ~ "OpenResty",
      TRUE ~ "Other"
    )
```

Figure 2.3.2.2

Technical server details are transformed into meaningful categories that are easier to analyse. Servers are classified into Outdated, Modern, and Unknown based on version thresholds for IIS, Apache, and Nginx, Litespeed, and Openresty. This enables the analysis to focus on security relevance rather than raw technical information. Meanwhile, server names are standardised into consistent types such as Apache, IIS, Nginx, LiteSpeed, and OpenResty, while very small categories are grouped into "Other". These transformations directly support the study objectives "*Analysis Question 1: Do victims using outdated WebServer versions pay significantly higher Ransom compared to those using modern ones?*" and "*Analysis Question 2: Is there a significant variance in Ransom amounts across different WebServer types?*".

```
    ) %>%
    mutate(
      Server_Status = factor(Server_Status, levels = c("Outdated", "Modern", "Unknown"))
    ) %>%
    add_count(Clean_Type, name = "type_n") %>%
    mutate(Clean_Type = if_else(type_n < 30, "Other", Clean_Type)) %>%
    select(-type_n)
}
```

Figure 2.3.2.3

This code will be used to enhance the reliability and clarity of the analysis. Server types with very small sample sizes will be grouped into *"Other"* category to prevent unstable averages and misleading variation during statistical testing. These adjustments make the analysis more

robust and help guarantee that the comparisons of ransom amounts among server status and server types can generate meaningful and accurate results regarding whether certain web server characteristics are linked to higher ransomware demands.

2.3.3 Data Analysis

```
> # Skewness check (Raw USD)
> print(skewness(df_ws_clean$Ransom))
[1] 6.488591
>
> # Shapiro-Wilk (Raw USD)
> set.seed(123)
> sample_data <- df_ws_clean %>% sample_n(min(nrow(df_ws_clean), 5000))
> print(shapiro.test(sample_data$Ransom))

        Shapiro-Wilk normality test

data:  sample_data$Ransom
W = 0.53271, p-value < 0.00000000000000022
```

Figure 2.3.3.1

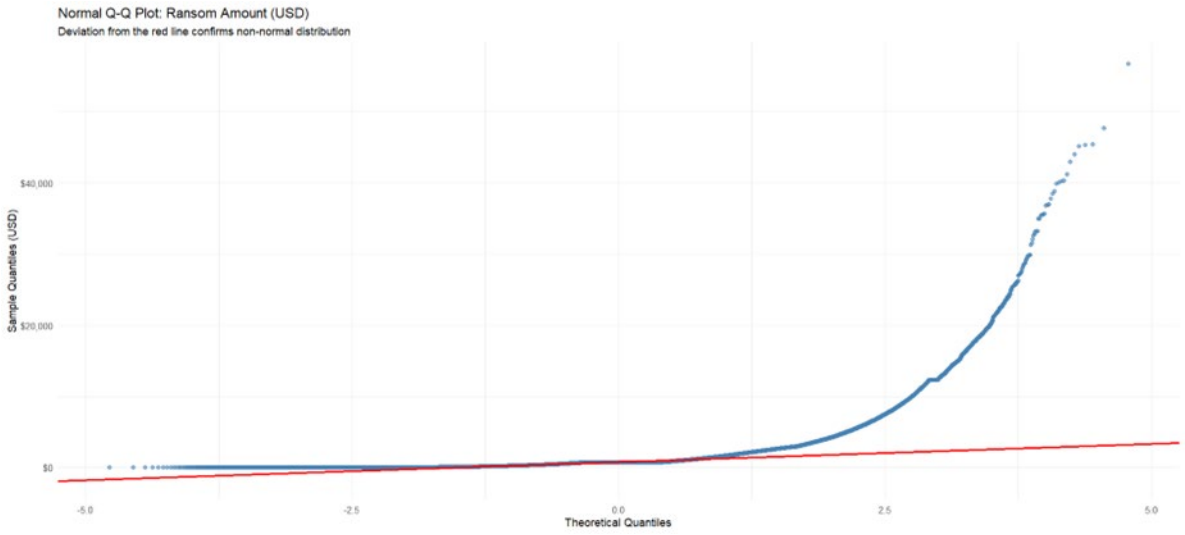


Figure 2.3.3.2

The skewness and Shapiro–Wilk tests are used to performed to test the hypothesis of whether the ransom amounts are distributed normally. This is important because most of the commonly used statistic test like t-tests and ANOVA will assume that the test is normally distributed. Especially in cybersecurity datasets, ransom amounts are usually not evenly distributed, where many small amount and suddenly goes to a extremely large amount. Therefore, it is important to test the distribution before comparing ransom values between different web server statuses and server types.

According to the result shown in Figure 2.3.4.1, the skewness value of 6.49 indicates it is highly right-skewed (positive). This means that most ransom amount is relatively low with only a few cases being extremely high which will push the distribution to the right. This reflects that the data is not balanced about the average value, thus the Shapiro–Wilk test supports this finding. The test is used to test the difference between data and normal distribution, where the null hypothesis assumes normality. With the results showing **W = 0.53271 with p-value < 0.00000000000000022**, is far below the common significance level of 0.05. Therefore, the null hypothesis of normality is rejected, indicating that the ransom data is not normally distributed.

Based on these results, we know that using non-parametric method like Mann-Whitney U test and Kruskal-Wallis test will be the best suitable when comparing ransom amounts between outdated and modern servers across different web server types. This is because using parametric tests that rely on normality will not be suitable to test raw ransom data. This approach helps to confirm that *Analysis Question 1* and *Analysis Question 2* are reliable and statistically valid.

In Figure 2.3.4.1, the pronounced upward deviation of the sample points from the red reference line at the upper part of the distribution (theoretical quantiles > 2) clearly indicates strong right-skewness. This pattern indicates that the distribution does not have a normal distribution, and the upper tails are heavier, reflecting the presence of extreme high-value observations. The high frequency of data points being concentrated in the lower end of the y-axis (close to $0) shows high frequency of relatively small ransom amounts. Whereas the sharp upward deviation of the final observations shows that there are really extreme high-value cases that contribute significantly to the observed skewness value of 6.49. This visual pattern observed in Q-Q plot with the fact that the positive deviation at the upper quantiles is strong, gives a very clear graphical explanation to the rejection of normality in the Shapiro-Wilk test and the large skewness coefficient in the data.

```
set.seed(123)
max_n <- 50000

df_q1_sample <- df_q1 %>%
  group_by(Server_Status) %>%
  group_modify(~ {
    n_rows <- nrow(.x)
    n_sample <- min(n_rows, max_n)
    slice_sample(.x, n = n_sample)
  }) %>%
  ungroup()
```

Figure 2.3.3.3

This code is used to create a balanced and manageable sample before analyzing the differences in ransom amounts between server statuses. The data is grouped by Server_Status first, and then sampling is carried out within each group. This avoids the dominance of a single category over the analysis that contain more records like the Modern servers as an example. Consequently, the outdated and the modern servers are compared in a fair and meaningful way in *Analysis Question 1*.

The *max_n <- 50000 line* sets as a maximum limit on how many records can be taken from each group. This helps to reduce processing time and computational load while still keeping a large sample to reflect the overall data pattern. On the other hand, set.seed(123) ensures that the sampling process is reproducible. This means that every time the analysis runs, the same records are selected, allowing results to remain consistent and easier to verify.

Analysis Question 1: Do victims using outdated WebServer versions pay significantly higher Ransom compared to those using modern ones?

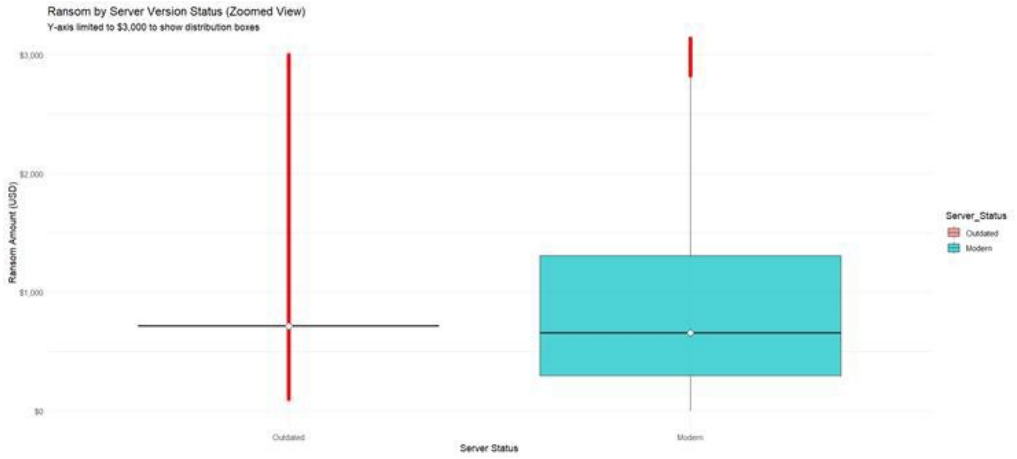


Figure 2.3.3.4

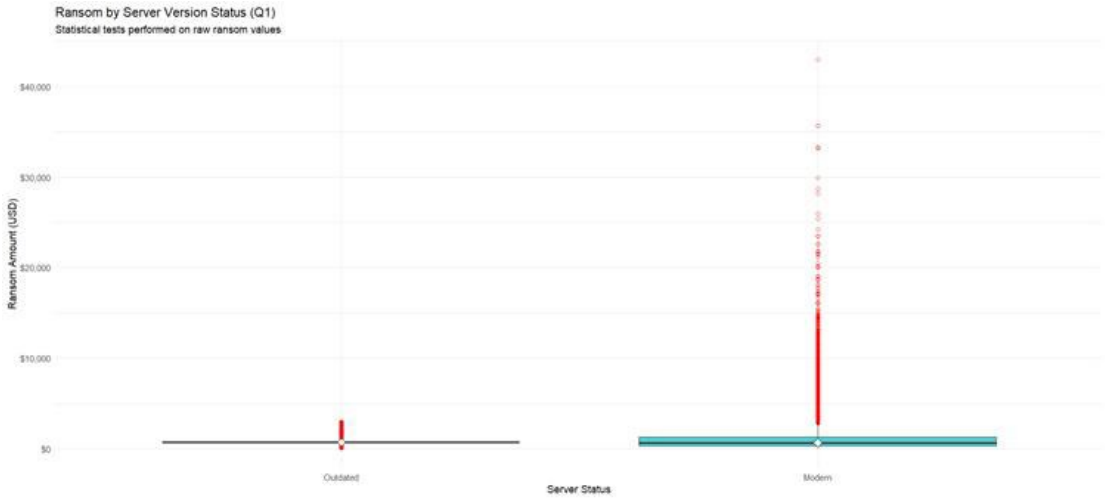


Figure 2.3.3.5

Figure 2.3.3.4 shows the Zoomed version of the Boxlot (for clearer view of the graph)

Figure 2.3.3.5 shows the Orginal version of the Boxlot

Distribution and typical ransom Amounts

The boxplot shows that both outdated and modern servers have a quite similar ransom amount approximately between $800 and $900. This suggests that many victims face there a common baseline demand according to server type. Modern servers have a wider interquartile range (IQR), showing greater variance in ransom amounts compared to outdated servers where payments tend to cluster within a smaller range. In simple terms, while the typical ransom may be similar, modern servers experience a wider spread of financial outcomes, including both moderate and very high demands (Hernandez-Castro et al., 2020).

Standardized vs Targeted Attacks

The tightly compressed distribution seen in the outdated server group indicates a more standardized price pattern. This is consistent with the automated ransomware attacks targeting the specific vulnerabilities of older versions of software. This type of attack is typically performed on large scale, with similar transactions on many victims, which results in similar ransom demands. Conversely, the widely distribution of ransom values for modern servers indicates that the attacks could be highly targeted. Modern systems are usually harder to compromise, therefore when attacks are successfully made, means that they made a lot of effort or manual activity, thus higher ransom are demanded from the attackers based on victim's size, data value, or expected ability to pay (Halgamuge, 2024).

High-Value Targets and Outliers

Larger number of high-value outliers are more common in the modern server group may potentially mean that attackers target organizations that they view as a more valuable or better resourced. Modern server environments may be linked to larger or more established organizations and this may encourage attackers to demand higher payments after gaining access (Paquet-Clouston et al., 2019). As a result, abnormally high ransom amounts appear more frequently in this category, going far beyond the normal baseline in more opportunistic attacks. Generally, this pattern indicated that the size of ransom are not only influenced by server version, but also varies on the attacking strategies.

Analysis Question 2: Is there a significant variance in Ransom amounts across different WebServer types?

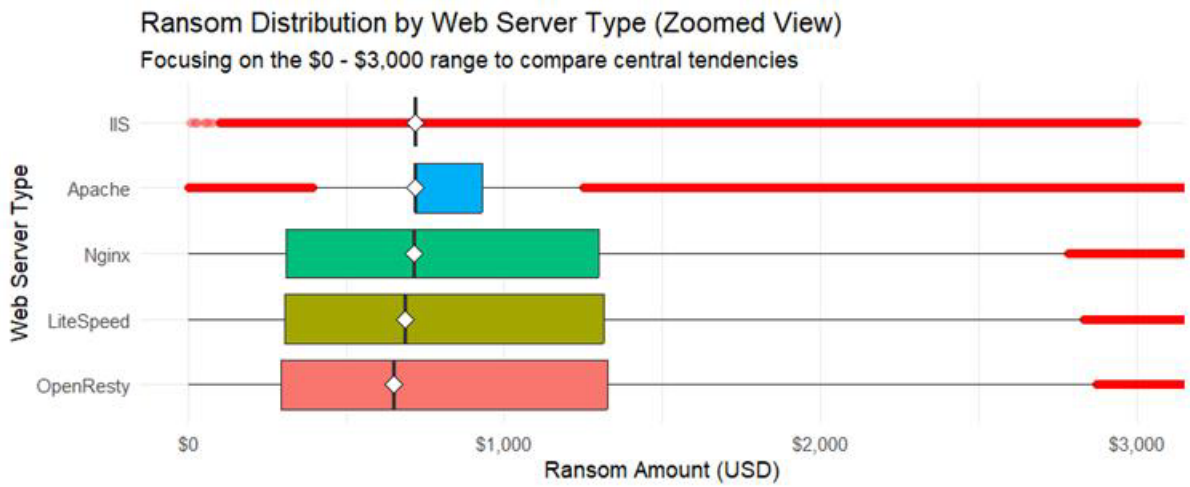


Figure 2.3.3.6

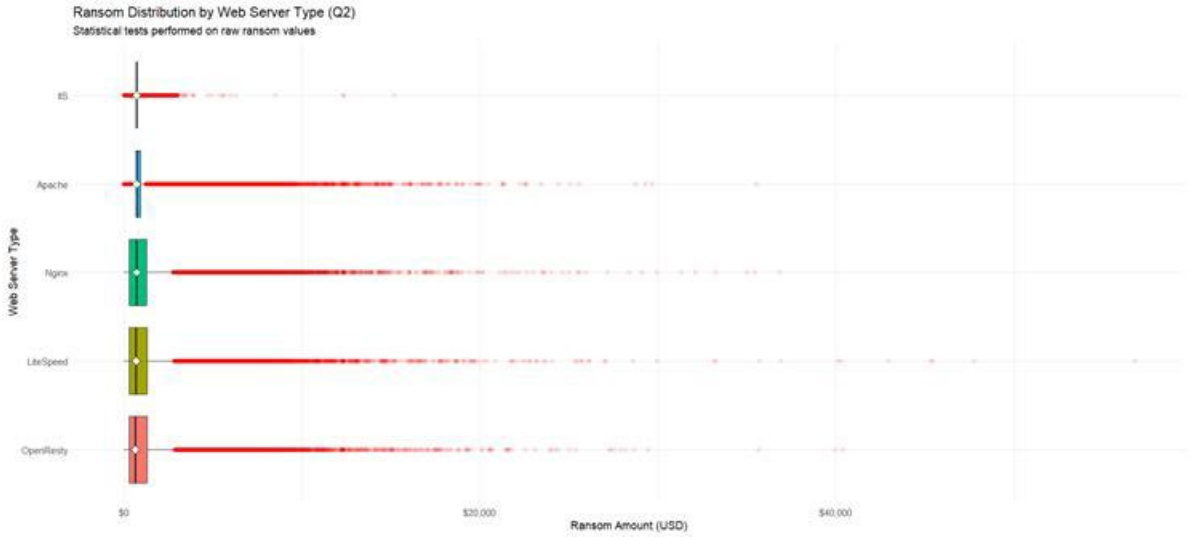


Figure 2.3.3.7

Figure 2.3.3.6 shows the Zoomed version of the Boxlot (for clearer view of the graph)

Figure 2.3.3.7 shows the Orginal version of the Boxlot

Typical Ransom Amounts Across Server Types

From the graph, we can see that despite the variation in web server technologies, the median ransom values are quite similar, generally falling between $700 and $900. This implies that ransomware demand is inclined to have a common baseline. In other words, possible that the attackers starts the attacks with a standard amount initially regardless of whether the victim uses older systems like IIS or a newer platforms like OpenResty. The initial demand seems to be relatively consistent, with adjustments likely made after depending on the organization or situation.

Differences in Variation Between Server Types

Although the median values are similar, ransom amounts spreads differently noticeably between server types. IIS and Apache have much narrower boxes, meaning majority of the ransom payments fall within a smaller and more consistent range. This trend is frequently linked to automated ransomware attacks which exploit known vulnerabilities and apply similar demands across many victims. In contrast, Nginx, LiteSpeed, and OpenResty show larger interquartile ranges, indicating greater variation in ransom amounts. This suggest that the attacks involving such platforms can be sometimes more targeted, which means that the demands may vary depending on the size of the organization, the relevance of data, or operational impact.

High-Value Demands and Outliers

High-value outliers was present in many server types indicate that extremely high ransom demands are possible with or without the underlying technological type. However, wider distributions seen in platforms like Nginx and OpenResty may reflect situations where attackers aim to seek for higher payouts after finding potentially valuable targets. These patterns indicate that server technology is a factor but also ransom size is heavily dependent on the attacker strategy and value of the victim, rather than server type alone.

2.3.4 Hypothesis Formulation and Testing

Null Hypothesis (H_0):

There is **no significant relationship** between WebServer characteristics (Version Status and Server Type) and the amount of Ransom demanded from victims.

Alternative Hypothesis (H_1):

WebServer characteristics **significantly influence** Ransom amounts, where specific vulnerabilities such as outdated versions and certain server types are associated with higher financial demands.

Analysis Question 1: Do victims using outdated WebServer versions pay significantly higher Ransom compared to those using modern ones?

```
> q1_test <- df_q1_sample %>%
+   wilcox_test(Ransom ~ Server_Status) %>%
+   add_significance()
>
> q1_effect <- df_q1_sample %>%
+   wilcox_effsize(Ransom ~ Server_Status)
>
> q1_summary <- df_q1_sample %>%
+   group_by(Server_Status) %>%
+   get_summary_stats(Ransom, type = "median_iqr")
>
> print("--- Q1 Results ---")
[1] "--- Q1 Results ---"
> print(q1_test)
# A tibble: 1 × 8
  .y.     group1   group2    n1    n2  statistic         p p.signif
  <chr>   <chr>    <chr>  <int> <int>      <dbl>     <dbl> <chr>
1 Ransom Outdated Modern 25537 50000 738506512. 3.53e-278 ****
> print(q1_effect)
# A tibble: 1 × 7
  .y.     group1   group2 effsize    n1    n2 magnitude
* <chr>   <chr>    <chr>    <dbl> <int> <int> <ord>
1 Ransom Outdated Modern   0.130 25537 50000 small
> print(q1_summary)
# A tibble: 2 × 5
  Server_Status variable     n median   iqr
  <fct>         <fct>    <dbl>  <dbl> <dbl>
1 Outdated      Ransom   25537    717     0
2 Modern        Ransom   50000    659  1010
```

Figure 2.3.4.1

Statistical Significance vs. Real-World Impact

The Wilcoxon Rank-Sum test also known as Mann-Whitney U test shows the result with an extremely low p-value (3.53e-278), indicating that the variance between ransom amounts between outdated and modern server is statistically significant. The effect size is only 0.130 which is considered small. Relatively, the median ransom for outdated servers is $717 compared to $659 for modern servers with a difference of less than $60. This indicates that although outdated servers attract slightly higher baseline demands, server lifespan is just one of the factors among many that influence ransom amounts (Pym, 2025; Hernandez-Castro, Cartwright, & Stepanova, 2019).

Evidence of Standardized Ransom Demands in Outdated Systems

The interquartile range (IQR) for the outdated server group is 0, indicating that at least 50% of ransom demands cluster precisely around the median of $717. This high level of standardization supports the hypothesis of automated, standardized attacks. Outdated servers (e.g., IIS < 8 or Apache < 2.4) are often compromised via mass exploitation scripts, resulting in fixed, commodity-style ransom demands (Connolly, Wall, Lang, & Oddson, 2020). In this case, the ransom would be an indication of how easily the vulnerability could be exploited rather than the victim's value.

Targeted Attacks and High Variability in Modern Systems

By comparison, modern servers have a considerably higher IQR value of 1,010, reflecting a wide and less predictable range of ransom amounts. Even though the median is slightly lower ($659), there is a large variance which indicates that attacks on modern server are often more sophisticated. Once attackers bypass the stronger security defenses, they are likely to adjust their demands based on the targeted victim, rather than applying a fixed price (Connolly, Wall, Lang, & Oddson, 2020). This supports the "Whale Hunting" hypothesis, modern servers are often associated with higher-value targets, leading to more aggressive and variable ransom demands in the upper quartiles.

Conclusion

While victims using outdated servers do experience a statistically higher median ransom, the most important insight is the nature of the attack. Outdated systems are subjected to standardized, automated demands, whereas modern systems face highly variable and targeted extortion, reflecting a strategic adjustment by attackers based on the organizational value they recognized.

Analysis Question 2: Is there a significant variance in Ransom amounts across different WebServer types?

```
[1] "--- Q2 Summary by Server Type ---"
> print(server_stats)
# A tibble: 5 × 4
  Clean_Type  Count Mean_USD Median_USD
  <chr>       <int>    <dbl>      <dbl>
1 Apache     188717     992.       717.
2 IIS         29897     925.       717.
3 Nginx       79139    1101.       715.
4 LiteSpeed   76803    1111.       687.
5 OpenResty   74647    1107.       651.
```

Figure 2.3.4.2

```
> q2_test <- kruskal.test(Ransom ~ Clean_Type, data = df_q2)
> q2_effect <- df_q2 %>%
+   kruskal_effsize(Ransom ~ Clean_Type)
>
> print("--- Q2 Kruskal-Wallis Results ---")
[1] "--- Q2 Kruskal-Wallis Results ---"
> print(q2_test)

        Kruskal-Wallis rank sum test

data:  Ransom by Clean_Type
Kruskal-Wallis chi-squared = 3593.8, df = 4, p-value < 0.00000000000000022

> print(q2_effect)
# A tibble: 1 × 5
  .y.         n effsize method  magnitude
* <chr>   <int>   <dbl> <chr>   <ord>
1 Ransom 449203 0.00799 eta2[H] small
```

Figure 2.3.4.3

Statistical Significance and Null Hypothesis Rejection

The Kruskal-Wallis test calculated an extremely small **p-value** of **0.00000000000000022** ($< 2.2 \times 10^{-16}$), indicating that the probability of these differences occurring by chance is effectively zero. We can **reject the null hypothesis**, confirming that web server type (IIS, Apache, Nginx, LiteSpeed, or OpenResty) has a statistically significant effect on both distribution and scale of ransom demands. Or we could say that the type of server an organization uses depends on shaping the ransom amounts they are likely to face.

Magnitude of Differences Across Server Types

The Chi-squared statistic of 3593.8 with 4 degrees of freedom (representing the five server categories compared) is exceptionally high. In a Kruskal-Wallis test, a higher Chi-squared value indicates greater differences between the median ranks of the categories. This mathematically supports the patterns seen in the boxplots, where although median ransom amounts may appear similar across server types, the distribution and variability of values vary significantly. Legacy platforms like IIS tend to experience more uniform, standardized ransom demands, whereas modern stacks like OpenResty demonstrate higher deviation, indicating more unpredictable financial outcomes.

Practical Implications for Attacker Behavior

These findings suggest that the attackers use the “architectural signature” of a web server as an important signal when they target exploitation. The significant differences in distribution imply that not all attacks are treated the same. Common server types are targeted with automated, fixed-price attacks, while modern or high-value servers are more likely to face manual, high-stakes ransom demand. Essentially, the server type helps attackers estimate organizational value, leading them to shape both the scale and variability of ransom demands (Microsoft Security, 2022; Counter Ransomware Initiative, 2024).

Conclusion

The difference between ransom amount in various types of web servers is statistically significant. Although baseline ransom demands seem to be relatively standardized, their level

of variation difffers among servers. IIS servers are more likely to receive more steady, fixed price demands which are characteristic of automated attacks. Whereas Nginx and OpenResty are more varied, indicating that they are more targeted and potentially higher-value extortion attempts.

Sherwin A/L Jesudass TP075823

2.4.1 Research Objective & Analysis Question

Objective:

To analyse the influence of Notify and Encoding on the Ransom amount to identify which threat actors demand the highest payments

Analysis Questions:

- Which Notify is associated with the highest average Ransom?

- Does the type of Encoding used in the defacement correlate with the Ransom amount demanded?

2.4.2 Data Preparation

Step 1: Library and Data Loading

For this analysis, we used `HackingData_Cleaned 2.csv` as our main dataset. We started by loading the required R libraries `dplyr` for manipulating the data and `ggplot2` for plotting graphs and then read the CSV file into R using `read_csv`.

```
# Load necessary libraries
# To run this, ensure the packages are installed:
# install.packages(c("dplyr", "ggplot2", "readr", "nortest"))
library(dplyr)
library(ggplot2)
library(readr)
```

```
> # Load necessary libraries
> # To run this, ensure the packages are installed:
> # install.packages(c("dplyr", "ggplot2", "readr", "nortest"))
> library(dplyr)

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union

> library(ggplot2)
> library(readr)
```

```
# Data Preparation
# 1. Load the dataset
file_path <- "HackingData_Cleaned 2.csv"
hacking_data <- read_csv(file_path, show_col_types = FALSE)
```

```
> # 1. Load the dataset
> file_path <- "HackingData_Cleaned 2.csv"
> hacking_data <- read_csv(file_path, show_col_types = FALSE)
```

Step 2: Data Cleaning (Duplicate Removal)

First, we wanted to make sure we didn't have any repeated records that could mess up our stats, so we removed any exact duplicates using `distinct()`. This ensures that we aren't counting the same attack twice.

```
# Remove duplicates
hacking_data_clean <- hacking_data %>% distinct()
```

```
> # Remove duplicates
> hacking_data_clean <- hacking_data %>% distinct()
```

Step 3: Variable Selection and Transformation

We narrowed our dataset down to just the columns we needed `Notify`, `Encoding`, and `Ransom`. Upon checking the `Ransom` column, we noticed the values were log-transformed. To get a sense of the actual money involved, we converted these values back to their original scale using `expm1()` which reverses the natural log so we could work with real dollar amounts. We also made sure `Notify` and `Encoding` were treated as categories (factors) for the analysis.

```
# 3. Variable Selection and Type Conversion
# Convert Ransom from log scale back to numeric value
analysis_data <- hacking_data_clean %>%
  select(Notify, Encoding, Ransom) %>%
  mutate(
    Notify = as.factor(Notify),
    Encoding = as.factor(Encoding),
    Ransom = expm1(as.numeric(Ransom)) # Convert log-scale back to real money
  ) %>%
  filter(!is.na(Ransom)) # Ensure no missing Ransom values
```

```
> # 3. Variable Selection and Type Conversion
> # Convert Ransom from log scale back to numeric value
> analysis_data <- hacking_data_clean %>%
+   select(Notify, Encoding, Ransom) %>%
+   mutate(
+     Notify = as.factor(Notify),
+     Encoding = as.factor(Encoding),
+     Ransom = expm1(as.numeric(Ransom)) # Convert log-scale back to real money
+   ) %>%
+   filter(!is.na(Ransom)) # Ensure no missing Ransom values
```

Step 4: Outlier Identification (IQR Method)

Finally, we checked for outliers in the ransom amounts using the Interquartile Range (IQR) method. We decided to keep these outliers in our final analysis. A huge ransom isn't necessarily a mistake it's often a major attack, which is exactly what we want to study. Since we are using rank-based tests later Kruskal-Wallis, having these extreme values won't break our analysis, but identifying them helps us understand the spread of the data.

Calculate Quartiles and IQR

```
# 4. Outlier Handling
summary(analysis_data$Ransom)

Q1 <- quantile(analysis_data$Ransom, 0.25, na.rm = TRUE)
Q3 <- quantile(analysis_data$Ransom, 0.75, na.rm = TRUE)
IQR_value <- IQR(analysis_data$Ransom, na.rm = TRUE)
```

```
> # 4. Outlier Handling
> summary(analysis_data$Ransom)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
      1     447     717    1043    1132   56781
> Q1 <- quantile(analysis_data$Ransom, 0.25, na.rm = TRUE)
> Q3 <- quantile(analysis_data$Ransom, 0.75, na.rm = TRUE)
> IQR_value <- IQR(analysis_data$Ransom, na.rm = TRUE)
```

Define bounds

```
lower_bound <- Q1 - 1.5 * IQR_value
upper_bound <- Q3 + 1.5 * IQR_value
```

```
> lower_bound <- Q1 - 1.5 * IQR_value
> upper_bound <- Q3 + 1.5 * IQR_value
```

Identify outliers for reporting only

```
# Flagging outliers
outliers <- analysis_data %>%
  filter(Ransom < lower_bound | Ransom > upper_bound)

cat("Number of outlier transactions detected:", nrow(outliers), "\n")
cat("Percentage of dataset:", round((nrow(outliers)/nrow(analysis_data))*100, 2), "%\n")
```

```
> # Flagging outliers
> outliers <- analysis_data %>%
+   filter(Ransom < lower_bound | Ransom > upper_bound)
> cat("Number of outlier transactions detected:", nrow(outliers), "\n")
Number of outlier transactions detected: 61668
> cat("Percentage of dataset:", round((nrow(outliers)/nrow(analysis_data))*100, 2), "%\n")
Percentage of dataset: 10.73 %
```

2.4.4 Data Analysis

Analysis Question 1: Which Notify is associated with the highest average Ransom?

To support our first objective of identifying which threat actors allow for the highest financial gains, we grouped the data by Notify and calculated the mean and median ransom for each group. We looked at both because mean values can be easily skewed by massive ransoms, so the median gives us a good backup check.

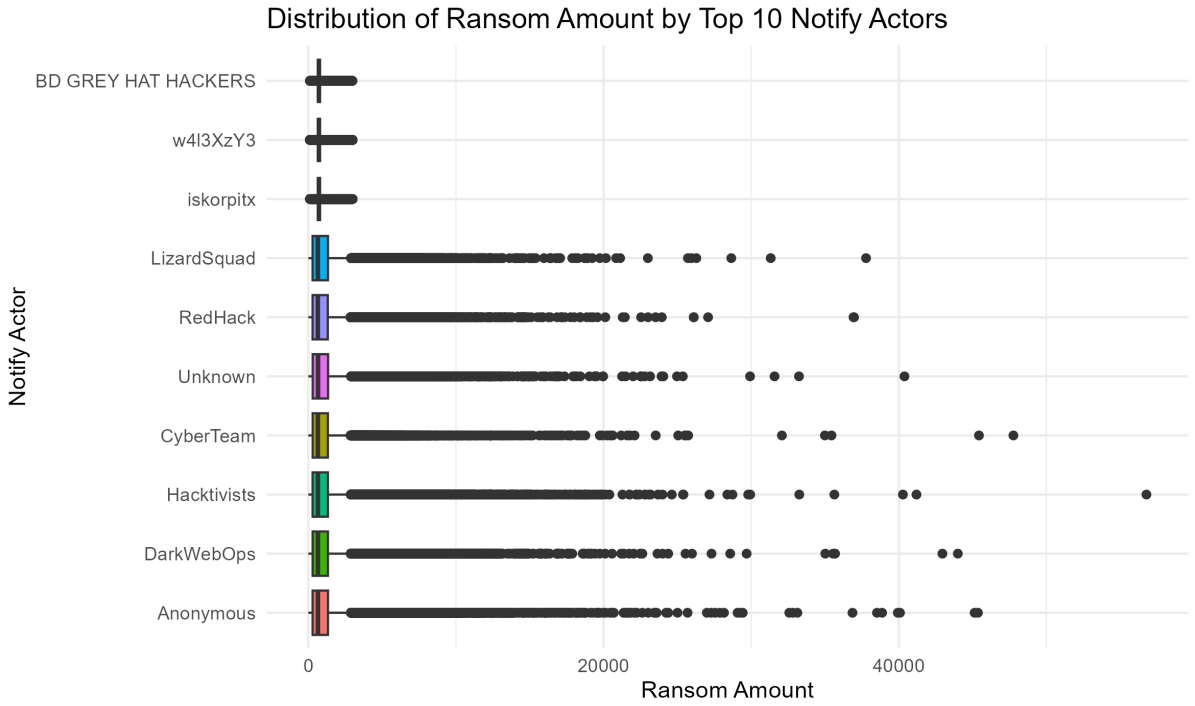
```
# Summary Statistics
notify_summary <- analysis_data %>%
  group_by(Notify) %>%
  summarise(
    Count = n(),
    Mean_Ransom = mean(Ransom, na.rm = TRUE),
    Median_Ransom = median(Ransom, na.rm = TRUE),
    SD_Ransom = sd(Ransom, na.rm = TRUE),
    Max_Ransom = max(Ransom, na.rm = TRUE)
  ) %>%
  arrange(desc(Mean_Ransom))

print(head(notify_summary, 10))
```

```
> # Summary Statistics
> notify_summary <- analysis_data %>%
+   group_by(Notify) %>%
+   summarise(
+     Count = n(),
+     Mean_Ransom = mean(Ransom, na.rm = TRUE),
+     Median_Ransom = median(Ransom, na.rm = TRUE),
+     SD_Ransom = sd(Ransom, na.rm = TRUE),
+     Max_Ransom = max(Ransom, na.rm = TRUE)
+   ) %>%
+   arrange(desc(Mean_Ransom))
> print(head(notify_summary, 10))
# A tibble: 10 × 6
   Notify           Count Mean_Ransom Median_Ransom SD_Ransom Max_Ransom
   <fct>            <int>       <dbl>         <dbl>     <dbl>       <dbl>
 1 DENIAL OF SERVICE     1       3000.         3000.        NA       3000.
 2 Crazy Cows            1       2996.         2996.        NA       2996.
 3 spewn                 1       2985.         2985.        NA       2985.
 4 GangstaKilla          1       2978.         2978.        NA       2978.
 5 Moroccanwolf          1       2978.         2978.        NA       2978.
 6 MakassarHack          1       2975.         2975.        NA       2975.
 7 supreme eternity      1       2975.         2975.        NA       2975.
 8 fa3al188              1       2974.         2974.        NA       2974.
 9 ?????? ??????        1       2972.         2972.        NA       2972.
10 hacked by artin       1       2972.         2972.        NA       2972.
>
```

This gave us a table of the top 10 most aggressive threat actors by average ransom. We also made a boxplot to visualize the spread of ransoms for these top groups.

```
# Visualization: Boxplot of Ransom by Notify
top_notify <- analysis_data %>%
  count(Notify, sort = TRUE) %>%
  head(10) %>%
  pull(Notify)

# Boxplot for Top 10 Notify
plot_notify <- analysis_data %>%
  filter(Notify %in% top_notify) %>%
  ggplot(aes(x = reorder(Notify, -Ransom), y = Ransom, fill = Notify)) +
  geom_boxplot() +
  coord_flip() +
  labs(
    title = "Distribution of Ransom Amount by Top 10 Notify Actors",
    x = "Notify Actor",
    y = "Ransom Amount"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

print(plot_notify)
```

```
> # Visualization: Boxplot of Ransom by Notify
> top_notify <- analysis_data %>%
+   count(Notify, sort = TRUE) %>%
+   head(10) %>%
+   pull(Notify)
> # Boxplot for Top 10 Notify
> plot_notify <- analysis_data %>%
+   filter(Notify %in% top_notify) %>%
+   ggplot(aes(x = reorder(Notify, -Ransom), y = Ransom, fill = Notify)) +
+   geom_boxplot() +
+   coord_flip() +
+   labs(
+     title = "Distribution of Ransom Amount by Top 10 Notify Actors",
+     x = "Notify Actor",
+     y = "Ransom Amount"
+   ) +
+   theme_minimal() +
+   theme(legend.position = "none")
> print(plot_notify)
```



Analysis Question 2: Does Encoding type correlate with Ransom?

To address our second objective of determining if specific technical methods correlate with higher financial demands, we did the same thing for `Encoding` to see if the technical method used for the defacement had any link to how much ransom was demanded.

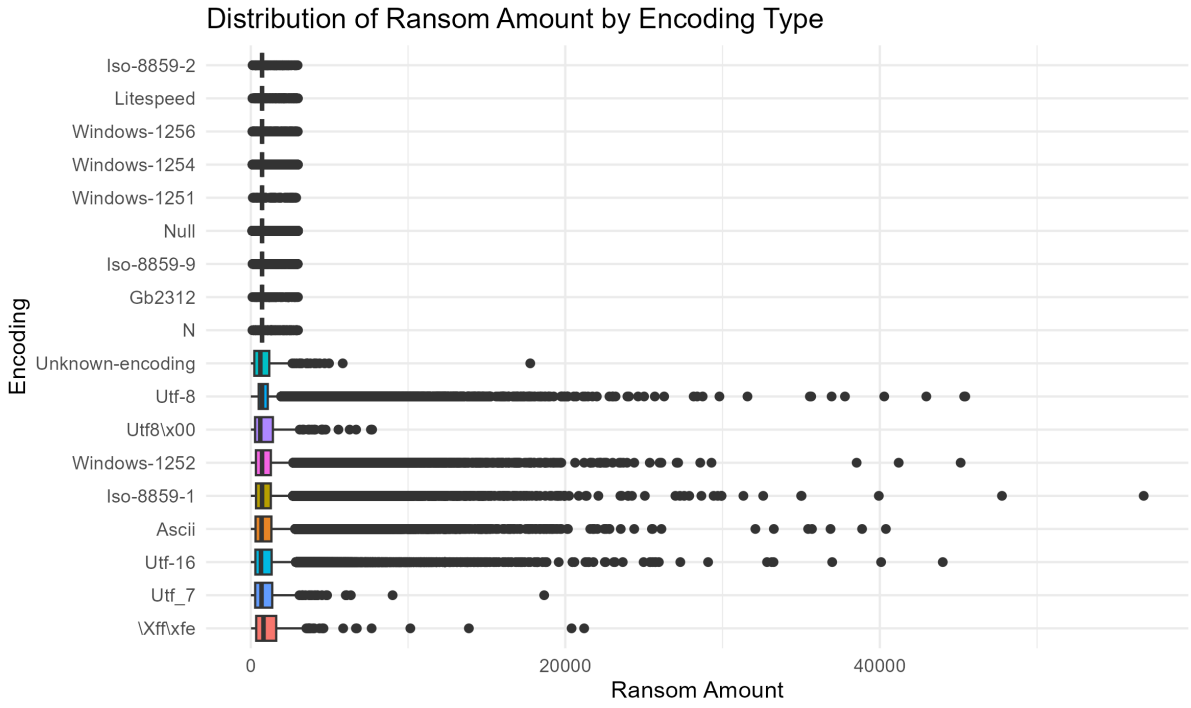
```r
# Summary Statistics
encoding_summary <- analysis_data %>%
  group_by(Encoding) %>%
  summarise(
    Count = n(),
    Mean_Ransom = mean(Ransom, na.rm = TRUE),
    Median_Ransom = median(Ransom, na.rm = TRUE)
  ) %>%
  arrange(desc(Mean_Ransom))

print(encoding_summary)
```

```
> # Summary Statistics
> encoding_summary <- analysis_data %>%
+   group_by(Encoding) %>%
+   summarise(
+     Count = n(),
+     Mean_Ransom = mean(Ransom, na.rm = TRUE),
+     Median_Ransom = median(Ransom, na.rm = TRUE)
+   ) %>%
+   arrange(desc(Mean_Ransom))
> print(encoding_summary)
# A tibble: 39 × 4
   Encoding       Count Mean_Ransom Median_Ransom
   <fct>          <int>       <dbl>         <dbl>
 1 "\\xff\\xfe"     250       1423.          808.
 2 "Windows-1250"     2       1335.         1335.
 3 "Utf_7"          272       1138.          683.
 4 "Utf-16"       74159       1106.          649.
 5 "Ascii"        77640       1103.          687.
 6 "Windows-874"     13       1093.          717.
 7 "Iso-8859-1"   83335       1092.          717.
 8 "Windows-1252" 83639       1088.          717.
 9 "Tis-620"          5       1083.          717.
10 "Asmo-708"        14       1061.          717.
# ℹ 29 more rows
# ℹ Use `print(n = ...)` to see more rows
>
```

This table shows us the average ransoms for different encoding types. Just like with the actors, we visualized this with a boxplot to see if certain methods consistently led to higher payouts.

```r
# Visualization: Boxplot of Ransom by Encoding
plot_encoding <- analysis_data %>%
  group_by(Encoding) %>%
  filter(n() > 100) %>% # Filter out very rare encodings for cleaner plot
  ggplot(aes(x = reorder(Encoding, -Ransom), y = Ransom, fill = Encoding)) +
  geom_boxplot() +
  coord_flip() +
  labs(
    title = "Distribution of Ransom Amount by Encoding Type",
    x = "Encoding",
    y = "Ransom Amount"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

print(plot_encoding)
```

```
> # Visualization: Boxplot of Ransom by Encoding
> plot_encoding <- analysis_data %>%
+   group_by(Encoding) %>%
+   filter(n() > 100) %>% # Filter out very rare encodings for cleaner plot
+   ggplot(aes(x = reorder(Encoding, -Ransom), y = Ransom, fill = Encoding)) +
+   geom_boxplot() +
+   coord_flip() +
+   labs(
+     title = "Distribution of Ransom Amount by Encoding Type",
+     x = "Encoding",
+     y = "Ransom Amount"
+   ) +
+   theme_minimal() +
+   theme(legend.position = "none")
> print(plot_encoding)
> ggsave("encoding_boxplot.png", plot = plot_encoding)
```

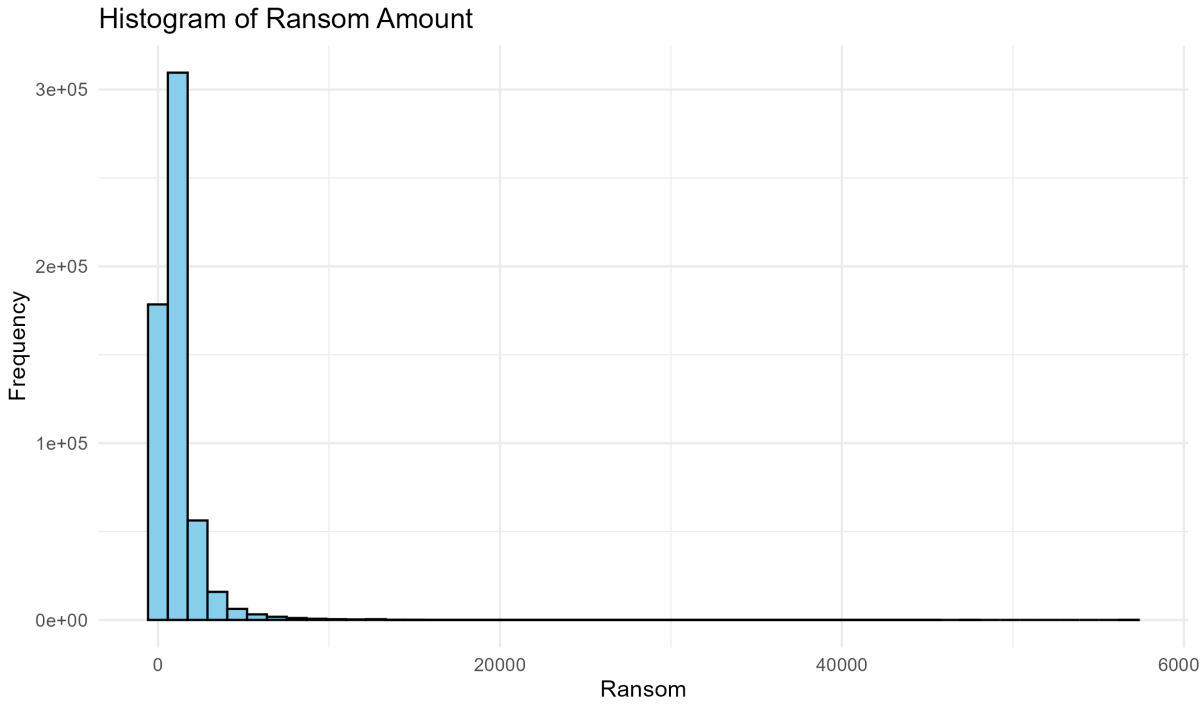


2.4.5 Hypothesis Formulation and Testing

Normality Testing

Before we could pick a statistical test, we had to check if our data followed a normal distribution. We started by plotting a histogram to visually inspect the shape of the data.

```
# 2.1.5 Hypothesis Formulation and Testing
# Normality Check
# Visual (Histogram) and Statistical (Shapiro-Wilk) checks

# 1. Visual Check: Histogram
plot_hist <- ggplot(analysis_data, aes(x = Ransom)) +
  geom_histogram(bins = 50, fill = "skyblue", color = "black") +
  labs(title = "Histogram of Ransom Amount", x = "Ransom", y = "Frequency") +
  theme_minimal()

print(plot_hist)
```




```
# 2. Statistical Check: Skewness
library(e1071)
ransom_skewness <- skewness(analysis_data$Ransom, na.rm = TRUE)
print(paste("Skewness:", ransom_skewness))
```

```
> # 2. Statistical Check: Skewness
> library(e1071)

Attaching package: 'e1071'

The following object is masked from 'package:ggplot2':

    element

> ransom_skewness <- skewness(analysis_data$Ransom, na.rm = TRUE)
> print(paste("Skewness:", ransom_skewness))
[1] "Skewness: 6.48909798222992"
```

The histogram showed a clear skew, suggesting the data wasn't normal. To confirm this statistically, we ran a Shapiro-Wilk test on a random sample of 5,000 records.

```
# 3. Statistical Check: Shapiro-Wilk (on random sample of 5000)
set.seed(123)
sample_ransom <- sample(analysis_data$Ransom, 5000)
shapiro_result <- shapiro.test(sample_ransom)
print(shapiro_result)
```

```
> # 3. Statistical Check: Shapiro-Wilk (on random sample of 5000)
> set.seed(123)
> sample_ransom <- sample(analysis_data$Ransom, 5000)
> shapiro_result <- shapiro.test(sample_ransom)
> print(shapiro_result)

        Shapiro-Wilk normality test

data:  sample_ransom
W = 0.47317, p-value < 2.2e-16
```

The p-value came back super small (less than 0.05), which confirms that our data is not normally distributed. because of this, we couldn't use standard tests like ANOVA, so we went with the Kruskal-Wallis Rank Sum Test, which works well for non-normal data with outliers.

Hypothesis Formulation

We came up with a single unified hypothesis to check if the attack details matter for the ransom price.

H0: The attack characteristics (Notify and Encoding Method) do not significantly affect the median Ransom amount.

H1: The attack characteristics (Notify and Encoding Method) significantly affect the median Ransom amount.

Test Results and Decision

To test this, we ran the Kruskal-Wallis test on both variables.

Test 1: Influence of Threat Actor (Notify)

```
kruskal_notify <- kruskal.test(Ransom ~ Notify, data = analysis_data)
print(kruskal_notify)
```

```
> kruskal_notify <- kruskal.test(Ransom ~ Notify, data = analysis_data)
> print(kruskal_notify)

        Kruskal-Wallis rank sum test

data:  Ransom by Notify
Kruskal-Wallis chi-squared = 13008, df = 9427, p-value < 2.2e-16
```

Result: The p-value was less than 0.05.

Test 2: Influence of Technical Method (Encoding)

```
kruskal_encoding <- kruskal.test(Ransom ~ Encoding, data = analysis_data)
print(kruskal_encoding)
```

```
> kruskal_encoding <- kruskal.test(Ransom ~ Encoding, data = analysis_data)
> print(kruskal_encoding)

        Kruskal-Wallis rank sum test

data:  Ransom by Encoding
Kruskal-Wallis chi-squared = 6040.4, df = 38, p-value < 2.2e-16
```

Result: The p-value was less than 0.05.

Since both p-values were below 0.05, we have enough evidence to reject the Null Hypothesis H0. This basically confirms that both the group behind the attack and the method they use (encoding) play a significant role in determining the ransom amount.

3.0 Group Hypothesis

3.1 Group Hypothesis Formulation

H0: There is no significance difference in the median Ransom amount across Notify, Encoding, WebServer version, Country, and DownTime

H1: The median Ransom amount varies significantly based on at least one of the independent variables, including Notify, Encoding, WebServer, Country, and DownTime.

3.2 Group Hypothesis Testing

Before we start performing the methodological that used to reject which hypothesis, we first choose a significant level for the overall test. We decided to use **0.05** as our significance level, it mean we require a 95% confidence level attackers financial demands.

After that, we first check the normality of the numerical variables to ensure that the suitable hypothesis testing technique.

```
> group_shapiro_results <- shapiro.test(sample(df$Ransom, 5000))
> print(group_shapiro_results)

        Shapiro-Wilk normality test

data:  sample(df$Ransom, 5000)
W = 0.53271, p-value < 0.00000000000000022
```

Figure 3. 1

By running the Shapiro-walk normality test for the numerical variable – Ransom, as per the results shown in the Figure 3.1, the p-value is significantly lower than the significance level. This provide the evidence that the Ransom data follow a non-normal distribution.

Due to its non-normal distribution, we have to use Kruskal-Wallis test to perform the hypothesis testing to evaluate the individual influence for each variable (Notify, Encoding, WebServer, Country, and DownTime).

```
> # Kruskal-Wallis Test Method
> test_notify <- kruskal.test(Ransom ~ Notify, data = df)
> test_encoding <- kruskal.test(Ransom ~ Encoding, data = df)
> test_server <- kruskal.test(Ransom ~ WebServer, data = df)
> test_country <- kruskal.test(Ransom ~ Country, data = df)
> test_downtime <- kruskal.test(Ransom ~ DownTime, data = df)
> # Summary Table
> group_results <- data.frame(
+   Variable = c("Notify", "Encoding", "WebServer", "Country", "DownTime"),
+   Chi_Squared = c(test_notify$statistic, test_encoding$statistic, test_server$statistic,
+                   test_country$statistic, test_downtime$statistic),
+   P_Value = c(test_notify$p.value, test_encoding$p.value, test_server$p.value,
+               test_country$p.value, test_downtime$p.value)
+ )
> view(group_results)
```

	Variable	Chi_Squared	P_Value
1	Notify	13014.436	0.0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000206759
5	DownTime	12341.108	0.0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
3	WebServer	10042.444	0.0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
2	Encoding	6045.156	0.0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000
4	Country	5817.483	0.0000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000000

Figure 3. 2

As per the Figure 3.2, every independent variable are tested by the Kruskal-Wallis testing and produced a p-value approaching zero ($p < 0.05$). Based on the pre-defined confidence level, we reject the Null hypothesis for all factors.

Beside that, we did observed another information from the test which is the high Chi-Squared value for Notify and DownTime. This means that, the incident reported by a person / group and the duration of the system unavailability is the most powerful composition for the prediction of ransom demands.

3.3 Overall Conclusion

From the results as shown in Figure 3.2, all independent variables had p-value less than the level of significance, which is 0.05. Therefore, the null hypothesis for all the factors was rejected, showing that all the independent factors are significant influence on the median ransom amount.

Furthermore, higher chi-squared values for Notify and DownTime were observed. The results of the Kruskal-Wallis test reflect that these variables play bigger role in the variation of ransom demand amounts than other variables. It is observed that notification of the crime and the unavailability of the system during crime play a critical role in the determination of the ransom demand by attackers.

Overall, the results validate the fact that ransom requests are not uniform and significantly influenced by multiple technical and various factors. Hence, the alternative hypothesis is supported.

4.0 Summary

4.1 Limitations and Recommendations

4.1.1 Limitations

There are several limitations that can be considered when interpreting results.

1. **Secondary data**: The analysis uses secondary data obtained from the existing dataset provided for academic purposes. There could be incomplete or missing values in this dataset that are out of control.
2. **Extreme values and skewness**: This dataset has extreme values and skewed distributions, which may affect the outputs and require the use of non-parametric statistical techniques.
3. **Limited variables and period**: The analysis is limited by the number of variables evaluated and the time period covered. This will cause failure of fully capture all factors that affect the results.
4. **Regional grouping**: Grouping countries into regions may lead to less accurate insights compared to analysis between countries.

4.1.2 Recommendations

1. **Secondary data:** Enhancing existing dataset with more sources like official statistics, recent reports, or updated datasets will be recommended to improve data reliability. Alongside, using cautious data to clean and impute methods can handle missing or inconsistent data before analysis.
2. **Extreme values and skewness:** Impact of extreme values can be reduced by using data transformations like logarithms or square roots. Besides, using non-parametric or resampling methods, which are less sensitive to skewed data could ensure results remain reliable.
3. **Limited variables and period:** It is recommended to expand the study by adding more relevant variables and extending the study period if possible. This will help capturing a more complete picture of the factors influencing the results. Sensitivity

checks can also be used to demonstrate the impact of how variables selection affects the findings.

4. **Regional grouping:** Analyzing country level data can have more precise insights. If using regional grouping technique, weighted averages or clustering methods that could better demonstrate regional difference could be considered to improve the analysis accuracy.

4.2 Workload Matrix & AI Usage Declaration

Area	Leong Yuan Kang TP076118	Sherwin A/L Jesudass TP075823	Lew Li Jun TP087450	Carrie Yap Liu Qing TP075883
1.0 Introduction	50%	16.67%	16.67%	16.67%
2.0 Analysis	25%	25%	25%	25%
3.0 Group Hypothesis	40%	10%	40%	10%
4.0 Summary	30%	10%	30%	30%

AI Usage Declaration

For this project, on certain part we did used AI in few area to optimize this project and generate out a most accurate finding.

1. Code Optimization: We used AI to assist in writing complex R script (use IP Address to get the country name) and to debug specific errors during the data preparation stage (regex WebServer version)

2. Statistical Justification: We used AI to provide guidance on how to interpret the content output from each section in order for us to better decide the next step

3. Documentation Refinement: We used AI to translate technical statistical output into a humanized and professional language that suitable for an academic report and presenting purposes.

All the AI-generated code was manually verified by each group members for accuracy against the provided datasets. The final interpretation and conclusion is reviewed and written by each group members to ensure the accurate of data findings.

5.0 References

The 10 Biggest Ransomware Payouts of the 21st Century» Admin By Request. (Brian Atkinson, August 2024). https://www.adminbyrequest.com/en/blogs/the-10-biggest-ransomware-payouts-of-the-21st-century

The 9 worst cyberattacks of 2015 (Paul Szoldra, pzdupe1, pzdupe2, Dec 2015)https://www.businessinsider.com/cyberattacks-2015-12

Cyber Warfare: Report on 2020 Shows Triple-Digit Increase Across All Malware Types (Glenn Rossman, Feb 2021) Cyber Warfare: Report on 2020 Shows Triple-Digit Increases Across All Malware Types

Hernandez-Castro, J., Cartwright, A., & Cartwright, E. (2020).
An economic analysis of ransomware and its welfare consequences. Royal Society Open Science, 7(3), 190023. https://doi.org/10.1098/rsos.190023

Halgamuge, M. N. (2024).
Ransomware reloaded: Re-examining its trend, research and mitigation in the era of data exfiltration. ACM Computing Surveys. https://doi.org/10.1145/

Paquet-Clouston, M., Haslhofer, B., & Dupont, B. (2019).
A tale of two markets: Investigating the ransomware payments economy. Communications of the ACM, 62(3), 71–79. https://doi.org/10.1145/3241037

Connolly, L. Y., Wall, D. S., Lang, M., & Oddson, B. (2020). An empirical study of ransomware attacks on organizations: An assessment of severity and salient factors affecting vulnerability. Journal of Cybersecurity, 6(1). https://doi.org/10.1093/cybsec/tyaa023

Hernandez-Castro, J., Cartwright, E., & Stepanova, A. (2019). Economic analysis of ransomware and its pricing strategies. ArXiv. https://arxiv.org/pdf/1703.06660.pdf

Pym, D. (2025). Dynamic pricing for ransomware (UCL Computer Science working paper). University College London. https://www0.cs.ucl.ac.uk/staff/D.Pym/ransomware-dynamic.pdf

Atlantic Council. (2020). Behind the rise of ransomware: Targeted extortion, big game hunting, and the evolution of attacks. Atlantic Council. Retrieved from https://www.atlanticcouncil.org/in-depth-research-reports/issue-brief/behind-the-rise-of-ransomware/

Schwartz, S. (2021, September 13). “Big game hunters”: Ransomware groups target their perfect victim. Cybersecurity Dive. Retrieved from https://www.cybersecuritydive.com/news/ideal-ransomware-victims-KELA/606253/

Counter Ransomware Initiative. (2024). Ransomware targeting framework (Disruption Working Group report). Australian Government Department of Home Affairs. https://www.homeaffairs.gov.au/foi/files/2024/fa-240901205-document-released.PDF